



SUMMER INTERNSHIP REPORT

on

AI-Driven Decision Intelligence for IPL Player Auction Analytics and Team Strategy Optimization

A Summer Research Internship carried out under



SARDAR PATEL TECHNOLOGY BUSINESS INCUBATOR

Sardar Patel Technology Business Incubator (SP-TBI)

SUBMITTED BY

Name	Roll No.
Anurag Kamble	2025510029
Shruti Suryawanshi	2025801016
Omkumar Sirmalwad	2025510061

Program / Department: CSE & MCA

Under the Guidance of

Prof. Saurabh N. Mehta

Vidyalankar Institute of Technology (VIT), Mumbai

Co-Guide

Prof. Kiran Talele

Sardar Patel Institute of Technology (SPIT), Mumbai

Academic Year: 2025–2026

Date of Submission: _____

CERTIFICATE

This is to certify that **Mr. Anurag Kamble, Ms. Shruti Suryawanshi, and Mr. Omkumar Sirmalwad** have successfully completed the **Summer Research Internship 2026** at **Sardar Patel Technology Business Incubator (SP-TBI)** on the project titled:

“AI-Driven Decision Intelligence for IPL Player Auction Analytics and Team Strategy Optimization.”

During the internship, the team worked on the development and analysis of two major components: **IPL auction price prediction using XGBoost** and a **quantile-based player ranking and classification framework**. The team contributed to data collection and preprocessing, feature engineering, model development and evaluation, player performance analysis, statistical classification, visualization, comparative analysis, and technical documentation.

The team also worked collaboratively on identifying and addressing issues such as **categorical encoding bias, temporal data leakage, model overfitting, bias-variance trade-offs, and appropriate player classification thresholds**. The members contributed to the preparation, analysis, documentation, and consolidation of the final internship report.

The work was carried out under the guidance of **Prof. Saurabh N. Mehta** and co-guidance of **Prof. Kiran Talele** during the **Summer Research Internship 2026**.

Date: _____

Place: Mumbai

Prof. Saurabh N. Mehta

Internal Guide

Prof. Kiran Talele

Co-Guide

Head of Department / Authorized Signatory

Plagiarism and AI Similarity Report

This page is reserved for the official plagiarism and AI-similarity verification required for the internship report. Replace the placeholders below with the actual values from the approved reports and attach the original report/screenshot if required.

Particular	Result / Details
Plagiarism Detection	
AI Similarity / AI Detection	
Software / Tool Used	
Date of Verification	
Report ID / Reference	

Note: The percentages must be taken from the actual institutional plagiarism/AI report and must not be estimated or manually invented.

Acknowledgement

We would like to express our sincere gratitude to Prof. Saurabh N. Mehta, CAO, Vidyalankar Institute of Technology, for guiding us through this internship and for pushing us to go back and question our own results instead of settling for numbers that merely looked good. His feedback at the review stage, in particular the suggestion to build a head-to-head player comparison alongside the ranking system, genuinely shaped the direction of this work and made it stronger.

We are equally grateful to Prof. Kiran Talele of Sardar Patel Institute of Technology for his technical mentorship and for making SP-TBI's resources available to us throughout the internship.

Our thanks to SP-TBI (Sardar Patel Technology Business Incubator) for organizing the Summer Research Internship 2026 and for giving us the space to work on a problem we found genuinely interesting rather than a prepackaged one.

Finally, we'd like to acknowledge each other. Building this across two studies, more debugging sessions than we can count, and a fair number of dead ends meant relying on each other constantly, and this report is as much a product of that collaboration as of any individual effort.

Abstract

This report documents the work carried out during the Summer Research Internship 2026, organized by the Sardar Patel Technology Business Incubator (SP-TBI), Mumbai, under the guidance of Prof. Saurabh N. Mehta (VIT) and co-guidance of Prof. Kiran Talele (SPIT). The internship ran under the broader theme "AI-Driven Decision Intelligence for IPL Player Auction Analytics and Team Strategy Optimization," looking at whether machine learning and decision-support techniques can help IPL franchises value players and plan auction strategy more systematically than scouting and raw statistics currently allow.

Two studies were carried out. Study 1 investigated an XGBoost-based auction price prediction model across three experiments. A baseline model evaluated on a random train-test split reported a test R^2 of 0.658, but this figure turned out to be inflated by temporal leakage — the split allowed the model to see players from years it was later tested on. A time-based split, holding out the 2017 season entirely, brought test R^2 down to 0.539, which is a more honest estimate of how the model would actually perform on a future auction. A third experiment added tree-depth restrictions and L1/L2 regularization to study the bias-variance tradeoff; this narrowed the train-test gap but also lowered test R^2 further, to 0.412, showing that the ceiling was set by the data available rather than by the model itself.

Study 2 built a quantile-based framework for ranking and classifying IPL batters and bowlers using ball-by-ball data from the 2012–2025 seasons. Batters were grouped into quintiles by batting average and bowlers into quartiles by wickets-per-match, rather than being scored on a single blended metric — an earlier composite z-score approach was tried first and abandoned because it consistently buried specialist bowlers under a scoring scheme that leaned toward batting.

Together, the two studies show both the promise and the limits of data-driven auction analytics: a carefully designed classification scheme can produce a defensible, transparent player ranking from data that's already available, while price prediction remains fundamentally constrained by what the auction dataset doesn't contain — retention history, franchise budgets, recent form, and injury status. This report covers the methodology, implementation, results, and the specific problems we ran into along the way for both studies.

Keywords: IPL Auction Analytics, XGBoost, Auction Price Prediction, Quantile Classification, Player Ranking, Sports Analytics, Decision Intelligence, Machine Learning.

Table of Contents

Certificate.....	02
Acknowledgement.....	04
Abstract.....	05
List of Figures.....	07
List of Tables.....	08
Abbreviations.....	10
Chapter 1: Introduction.....	11
Chapter 2: Organization Profile.....	12
Chapter 3: Internship Objectives and Planning.....	14
Chapter 4: Work Carried Out.....	16
Chapter 5: Technical Details.....	19
Chapter 6: Challenges and How We Solved Them.....	35
Chapter 7: Learning Outcomes.....	41
Chapter 8: Results and Discussion.....	42
Chapter 9: Conclusion and Future Scope.....	57
References.....	58
Appendices.....	59

List of Figures

Figure No.	Figure Name
Figure 4.1	IPL Auction Dataset Sample
Figure 4.2	Player Ranking and Classification Data Output
Figure 5.1	Regularized XGBoost Model Performance Output
Figure 5.2	Actual vs. Predicted Auction Price
Figure 5.3	Top 20 Features by Importance
Figure 5.4	Player Performance Matrix Used for Ranking
Figure 5.5	Composite Z-Score Player Ranking
Figure 5.6	Batting Ranking and Quintile Classification
Figure 5.7	Bowling Ranking and Quartile Classification
Figure 5.8	Batting Average Head-to-Head Comparison
Figure 5.9	Batting Strike Rate Head-to-Head Comparison
Figure 5.10	Bowling Average Head-to-Head Comparison
Figure 5.11	Economy Rate Head-to-Head Comparison
Figure 6.1	Model Performance Comparison Across Experiments
Figure 6.2	Composite Z-Score Ranking — Diagnostic View

Figure 6.3	Corrected Batting Tier Classification Output
Figure 8.1	Feature Importance and Leakage Analysis
Figure 8.2	Composite Z-Score Ranking (Supplementary)
Figure 8.3	Batting Performance Comparison
Figure 8.4	Bowling Performance Comparison
Figure 8.5	Cross-Metric Z-Score Patterns
Figure 8.6	Top-5 Batting Profile
Figure 8.7	Workload vs. Output
Figure 8.8	Batting Average Head-to-Head Heatmap
Figure 8.9	Batting Strike Rate Head-to-Head Heatmap

List of Tables

Table No.	Table Title
Table 3.1	Internship Timeline and Work Plan
Table 3.2	Tools and Technologies
Table 4.1	Datasets Used Across the Two Studies
Table 5.1	Class Definitions

Table 8.1	Experiment Comparison / Benchmarking
Table 8.2	Largest Prediction Errors, Experiment 1
Table 8.3	Batting Rank and Quintile Classification
Table 8.4	Bowling Rank and Quartile Classification
Table A.1	Data Dictionary (Key Fields)

Abbreviations

Sr. No.	Abbreviation	Full Form
1	AI	Artificial Intelligence
2	ML	Machine Learning
3	IPL	Indian Premier League
4	MCA	Master of Computer Applications
5	SPIT	Sardar Patel Institute of Technology
6	VIT	Vidyalankar Institute of Technology
7	SP-TBI	Sardar Patel Technology Business Incubator
8	WPM	Wickets Per Match
9	OHE	One-Hot Encoding
10	MAE	Mean Absolute Error
11	RMSE	Root Mean Squared Error
12	R^2	Coefficient of Determination
13	i.i.d.	Independent and Identically Distributed

Chapter 1: Introduction

1.1 About the Internship

This report is submitted for the Summer Research Internship 2026, organized by SP-TBI, Mumbai — an initiative of Bharatiya Vidya Bhavan's SPIT, affiliated with the Department of Science and Technology, Government of India. The internship was guided by Prof. Saurabh N. Mehta (VIT Mumbai) with Prof. Kiran Talele (SPIT) as co-guide, and carried out by a three-member team: Anurag Kamble, Shruti Suryawanshi, and Omkumar Sirmalwad. The theme was "AI-Driven Decision Intelligence for IPL Player Auction Analytics and Team Strategy Optimization." This report covers the two hands-on technical studies we actually built within that theme: an XGBoost price-prediction model, and a quantile-based player ranking framework.

1.2 Background and Motivation

An IPL franchise has to do several hard things at the same auction table at once — judge how good a player is, guess how they'll perform next season, keep the squad balanced, and not blow the budget doing it. Most teams are still leaning on the numbers everyone already has: batting average, strike rate, economy, scouting notes. None of that tells you much about how a player's value shifts with role, form, or what a specific squad happens to need. That's the gap we wanted to poke at.

There's a decent amount of prior research adjacent to this problem — decision-support frameworks for sport in general, cricket-specific ML for performance prediction, sequence models like Transformers and BERT, forecasting methods (ARIMA, ETS), ways of handling imbalanced data (SMOTE, cost-sensitive learning), team-selection optimization, and explainable/fair AI. Going through this literature, the pattern was clear: every piece has been validated somewhere, but usually in a different sport, or a different domain entirely. Nobody had actually put these pieces together into one cricket-specific, auction-aware system. That's what shaped the two studies in this report — build a defensible way to rank players using data that already exists, and honestly test how far a standard model can go in predicting what a player will actually sell for [1]–[10]. [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]

1.3 Problem Statement

- Auction price prediction: how accurately can a model predict a player's sale price from historical statistics, and what does the gap between an optimistic (random-split) evaluation and a realistic (time-based) one reveal about the actual difficulty of the problem?

- Player ranking and classification: given multi-season ball-by-ball data, how can batters and bowlers be ranked and grouped into tiers in a way that's fair with respect to how much they've played, robust to differences in scale, and transparent about which skill is actually driving the ranking?

1.4 Objectives

- Review the literature on sports decision-support and cricket ML to pin down the specific gap in IPL auction analytics.
- Build and iteratively debug an XGBoost auction-price-prediction model under both random-split and time-based validation.
- Diagnose the root causes of prediction error — data leakage, encoding bias, the bias-variance tradeoff — rather than just reporting the numbers.
- Design and implement a longevity-qualified, quantile-based classification framework for batters and bowlers.
- Write all of this up as a report that's honest about what worked, what didn't, and why.

1.5 Scope of Work

This report covers the two studies above in full. The wider literature-review theme — LSTM/ARIMA forecasting, SMOTE-based imbalance handling, NLP-based report summarization, optimization-based squad building — was surveyed as background to motivate the project, but building all of it out was outside what an internship of this length could cover. It's listed as future work in Chapter 9.

1.6 Organization of the Report

Chapter 2 profiles SP-TBI and the guiding institutes. Chapter 3 covers objectives, work plan, and tools. Chapter 4 describes the work carried out. Chapter 5 gives the technical detail for both studies. Chapter 6 is a dedicated account of the problems we hit and how we worked through them. Chapter 7 reflects on what we learned. Chapter 8 presents results. Chapter 9 concludes with future scope, followed by references and appendices.

Chapter 2: Organization Profile

2.1 Sardar Patel Technology Business Incubator (SP-TBI)

SP-TBI is a technology business incubator launched by Bharatiya Vidya Bhavan's SPIT, Mumbai, to help turn technology ideas into products. It was founded in 2015 as a not-for-profit trust affiliated with the Department of Science and Technology, Government of India. It draws student interns from SPIT and partner colleges and runs structured research

programs — including the Summer Research Internship 2026 — connecting students to real problems in data science, ML, decision intelligence, and web technologies.

2.2 Guiding Institutes

Vidyalankar Institute of Technology (VIT), Mumbai — represented by Prof. Saurabh N. Mehta, CAO, who provided overall research direction.

Sardar Patel Institute of Technology (SPIT), Mumbai — the co-guiding institute and parent of SP-TBI; Prof. Kiran Talele provided technical mentorship and facilitated incubator resources.

2.3 Department and Technology Focus

The internship sat within SP-TBI's data-science/AI research track. The stack was Python-based throughout: pandas and NumPy for data processing, scikit-learn and XGBoost for modeling, and Matplotlib/Seaborn for the visualizations in this report. [11], [12]

2.4 Team Structure

The team — Anurag Kamble, Shruti Suryawanshi, and Omkumar Sirmalwad — worked under Prof. Saurabh N. Mehta (guide) and Prof. Kiran Talele (co-guide).

Chapter 3: Internship Objectives and Planning

3.1 Objectives

- Build a reproducible pipeline turning raw ball-by-ball data into career-level batting and bowling figures for a defined player pool.
- Build and honestly evaluate an auction price prediction model, including under time-based conditions closer to what a franchise would actually face.
- Diagnose, rather than just report, any gap between optimistic and realistic model performance.
- Design a classification scheme that's statistically defensible and doesn't depend on arbitrary numeric cutoffs.

3.2 Methodology and Work Plan

Both studies followed the same loop: build a version, look at the output for anything that seemed off, work out why, and revise before moving on. That's reflected in the price-prediction study's three progressively corrected experiments (§5.2) and the ranking study's ten-step design history (§5.3.4) — neither one came out right on the first attempt, and the report tries to be honest about that rather than only showing the final version.

3.3 Timeline

Table 3.1 — Internship timeline and work plan

Phase	Focus Area	Key Output	Duration (Weeks)
1	Literature review of sports decision-support, cricket ML, forecasting, fairness/explainability	Related-work summary, identified research gap	Week 1–2
2	Data acquisition and cleaning — auction records (2008–2017), ball-by-ball data (2012–2025)	Cleaned, match-aggregated datasets	Week 3

3	Baseline XGBoost model; identified LabelEncoder ordinal bias	Experiment 1 results and root-cause diagnosis	Week 4
4	OneHotEncoder pipeline, feature engineering, time-based split	Experiment 2 results	Week 5–6
5	Regularization and bias-variance analysis; leakage review	Experiment 3 results	Week 7
6	Design and implementation of quantile classification pipeline	Batting and bowling leaderboards with tier classes	Week 8–9
7	Consolidation, documentation, report writing	Final technical reports and this internship report	Week 10

3.4 Tools and Technologies

Table 3.2: Tools and technologies

Category	Tools / Technologies	Purpose	Condition / Usage
Programming	Python 3.x	Core language for data processing and modeling	Used throughout both studies.
Machine Learning	XGBoost (regression), scikit-learn (train/test split, evaluation)	Auction price prediction experiments	Used for all three auction-price regression experiments.

Preprocessing	LabelEncoder, OneHotEncoder, ColumnTransformer	Handling categorical variables	Used for train/test preparation, preprocessing and evaluation.
Feature Engineering	Growth-rate features (RunGrowth, MatchGrowth, etc.)	Capturing player trajectory over seasons	Used in the baseline; replaced after ordinal-encoding bias was identified.
Ranking Models	Quantile-based classification (batter quintiles, bowler quartiles); composite z-score ranking	Player performance classification	Used in the improved categorical preprocessing pipeline.
Visualization	Matplotlib, Seaborn	Charts, heatmaps, radar plots	Used with OneHotEncoder to transform categorical feature groups.
Environment	Jupyter Notebook, VS Code	Development and experimentation	Used in Experiment 2 to capture year-over-year player trajectory.

Chapter 4: Work Carried Out

4.1 Literature Review and Research-Gap Identification

We started with a structured review of twenty prior works spanning sports decision-support frameworks, cricket-specific ML, Transformer/BERT-based sequence models, forecasting theory, imbalanced-data handling, team-selection optimization, and explainable/fair AI. This review — done jointly by all three of us — is what established that no existing system combines these techniques into one cricket-specific, auction-aware tool. That's the gap the two studies below try to address, at a much smaller scale than the full literature suggests is eventually possible. [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]

4.2 Data Acquisition and Preparation

Table 4.1 — Datasets used across the two studies

Dataset	Size	Notes / Issues	Used In
IPL Auction Dataset (2008–2017)	1,108 records, 42 features	Inconsistent fields across years; no retention info, budget caps, or demand signals	Study 1 — Price Prediction
Ball-by-Ball Delivery Records (2012–2025)	2012–2025 seasons	Aggregated to match level to avoid double-counting	Study 2 — Player Ranking
Longevity-Qualified Player Pool (2012–2025)	11 players	Appeared in every season 2012–2025, to keep comparisons fair	Study 2 — Player Ranking
Player Performance Matrix	11 qualifying players	Match-aggregated career figures used to compute batting/bowling metrics	Study 2 — Player Ranking
Composite Player Ranking	11 qualifying players	Standardized z-score ranking; supplementary, biased toward batting	Study 2 — Player Ranking

For Study 1, the historical auction records (2008–2017, 1,108 rows, 42 features) were used directly, since price prediction needs the actual sale price tied to each record. For Study 2, ball-by-ball records were used to reconstruct season-by-season appearances from the batter, bowler, and non-striker fields, from which the longevity-qualified pool — players who appeared in every season from 2012 to 2025 — was identified. Career figures came from match-level aggregation: we summed per-match totals, not raw delivery rows, specifically to avoid double-counting (see §6.1 for why that mattered).

4.3 Study 1 — Building and Debugging the Price Prediction Model

- Experiment 1: baseline XGBoost with LabelEncoder for categoricals and a random 80:20 split. [12]

- Experiment 2: switched to a OneHotEncoder / ColumnTransformer pipeline, added 7 engineered growth-rate features, and moved to a time-based split (train on 2008–2016, test on 2017). [12]
- Experiment 3: added tree-depth capping and L1/L2 regularization to study the bias-variance tradeoff. [12]

Each experiment's MAE, RMSE, and R^2 were logged and compared, and we checked feature importances after every run to look for leakage — the Status feature in particular (see §5.2.4 and §6.4).

#	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T
1	Id	Name	Year	Final Price	Role	Nationality	Team	Ent	Age	Matches	LMatches	Runs	LRuns	HS	LHS	Ave	Lave	StrRate	LStrRate	R
2	1	AB de Villiers	2008	12040000	Wicketkeeper batsman	South African	DD	0	24	6	0	95	0	26	0	19	0	96.93	0	
3	1	AB de Villiers	2009	14736000	Wicketkeeper batsman	South African	DD	0	25	15	6	465	95	105	26	51.66	19	130.98	96.93	
4	1	AB de Villiers	2010	13887000	Wicketkeeper batsman	South African	DD	0	26	7	15	111	465	45	105	15.85	51.66	93.27	130.98	
5	1	AB de Villiers	2011	50600000	Wicketkeeper batsman	South African	RCB	0	27	16	7	312	111	65	45	34.66	15.85	128.39	93.27	
6	1	AB de Villiers	2012	55297000	Wicketkeeper batsman	South African	RCB	0	28	16	16	319	312	64	65	39.87	34.66	161.11	128.39	
7	1	AB de Villiers	2013	58597000	Wicketkeeper batsman	South African	RCB	0	29	14	16	360	319	64	64	36	39.87	164.38	161.11	
8	1	AB de Villiers	2014	95000000	Wicketkeeper batsman	South African	RCB	0	30	14	14	395	360	89	64	35.9	36	158.63	164.38	
9	1	AB de Villiers	2015	95000000	Wicketkeeper batsman	South African	RCB	0	31	16	14	513	395	133	89	46.63	35.9	175.08	158.63	
10	1	AB de Villiers	2016	95000000	Wicketkeeper batsman	South African	RCB	0	32	16	16	687	513	129	133	52.84	46.63	168.79	175.08	
11	1	AB de Villiers	2017	95000000	Wicketkeeper batsman	South African	RCB	0	33	9	16	216	687	89	129	27	52.84	132.51	168.79	
12	2	Aaron Finch	2010	13887000	Batsman	Australian	RR	1	24	1	0	21	0	21	0	21	0	100	0	
13	2	Aaron Finch	2011	13800000	Batsman	Australian	DD	0	25	6	1	47	21	25	21	7.83	21	102.17	100	
14	2	Aaron Finch	2012	15081000	Batsman	Australian	DD	0	26	2	6	55	47	30	25	27.5	7.83	107.84	102.17	
15	2	Aaron Finch	2013	26635000	Batsman	Australian	PWI	0	27	14	2	456	55	67	30	32.57	27.5	135.71	107.84	
16	2	Aaron Finch	2014	40000000	Batsman	Australian	SRH	0	28	13	14	309	456	88	67	28.09	32.57	117.49	135.71	
17	2	Aaron Finch	2015	32000000	Batsman	Australian	MI	0	29	3	13	23	309	10	88	11.5	28.09	69.69	117.49	
18	2	Aaron Finch	2016	10000000	Batsman	Australian	GL	0	30	13	3	393	23	74	10	39.3	11.5	131.43	69.69	
19	2	Aaron Finch	2017	10000000	Batsman	Australian	GL	0	31	13	13	300	393	72	74	25	39.3	163.04	131.43	
20	3	Abhimanyu Mithun	2010	8000000	Bowler	Indian	RCB	0	21	1	1	5	0	5	0	5	0	125	0	
21	3	Abhimanyu Mithun	2011	11960000	Bowler	Indian	RCB	0	22	12	1	27	5	11	5	9	5	135	125	
22	3	Abhimanyu Mithun	2012	11960000	Bowler	Indian	RCB	0	23	1	12	0	27	0	11	0	9	0	135	
23	3	Abhimanyu Mithun	2013	11960000	Bowler	Indian	RCB	0	24	1	1	2	0	2	0	2	0	100	0	
24	3	Abhimanyu Mithun	2015	30000000	Bowler	Indian	MI	0	26	0	0	0	0	0	0	0	0	0	0	
25	3	Abhimanyu Mithun	2016	30000000	Bowler	Indian	SRH	0	27	0	0	0	0	0	0	0	0	0	0	
26	4	Abhishek Navar	2008	12000000	All-rounder	Indian	MI	0	25	14	0	206	0	45	0	25.75	0	149.27	0	

#	U	V	W	X	Y	Z	AA	AB	AC	AD	AE	AF	AG	AH	AI	AJ	AK	AL	AM	AN	AO	AP	AQ	AR	AS	AT
1	Lfifties	Hundreds	LHundreds	Fours	LFours	Sixes	LSixes	Catches	LCatches	Stumps	LStumps	Wkts	LWkts	Econ	LEcon	FourWkts	LFourWkts	FiveWkts	LFiveWkts	Indian	Specialist	Status				
2	0	0	0	5	0	1	0	3	0	0	0	0	0	0	0	0	0	0	0	0	0	1	218			
3	0	1	0	39	5	12	1	13	3	0	0	0	0	0	0	0	0	0	0	0	1	329				
4	3	0	1	7	39	0	12	6	13	0	0	0	0	0	0	0	0	0	0	0	1	895				
5	0	0	0	21	7	14	0	7	6	2	0	0	0	0	0	0	0	0	0	0	1	1316				
6	2	0	0	26	21	15	14	4	7	5	2	0	0	0	0	0	0	0	0	0	1	1075				
7	3	0	0	34	26	15	15	8	4	0	5	0	0	0	0	0	0	0	0	0	1	3764				
8	2	0	0	26	34	24	15	7	8	0	0	0	0	0	0	0	0	0	0	0	1	4415				
9	3	1	0	60	26	22	24	8	7	0	0	0	0	0	0	0	0	0	0	0	1	5281				
10	2	1	1	57	60	37	22	19	8	0	0	0	0	0	0	0	0	0	0	0	1	12131				
11	6	0	1	12	57	16	37	6	19	0	0	0	0	0	0	0	0	0	0	0	1	7467				
12	0	0	0	0	0	1	0	0	0	0	0	0	0	0	4.28	0	0	0	0	0	1	105				
13	0	0	0	3	0	3	1	4	0	0	0	0	0	0	6.5	4.28	0	0	0	0	1	441				
14	0	0	0	8	3	0	3	0	4	0	0	0	0	0	6.5	0	0	0	0	0	1	28				
15	0	0	0	54	8	16	0	2	0	0	0	0	1	0	12.25	0	0	0	0	0	1	2478				
16	4	0	0	31	54	9	16	6	2	0	0	0	1	0	12.25	0	0	0	0	0	1	2949				
17	2	0	0	4	31	0	9	0	6	0	0	0	0	0	0	0	0	0	0	0	1	3720				
18	0	0	0	45	4	11	0	3	0	0	0	0	0	0	0	0	0	0	0	0	1	5282				
19	5	0	0	25	45	19	11	5	3	0	0	0	0	0	0	0	0	0	0	0	1	4635				
20	0	0	0	1	0	0	0	0	0	0	0	0	1	0	13	10.5	0	0	0	0	1	77				
21	0	0	0	3	1	1	0	7	0	0	0	5	1	9.44	13	0	0	0	0	0	1	926				
22	0	0	0	0	3	0	1	0	7	0	0	0	5	11	9.44	0	0	0	0	0	1	1235				
23	0	0	0	0	0	0	0	0	0	0	0	1	0	9.25	11	0	0	0	0	0	1	561				
24	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	455				
25	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	621				
26	0	0	0	16	0	9	0	4	0	0	0	1	0	9.71	0	0	0	0	0	0	1	86				
27	0	0	0	14	16	7	9	4	4	0	0	4	1	7	9.71	0	0	0	0	0	1	0	272			
28	0	0	0	4	14	2	7	1	4	0	0	0	4	0	7	7	0	0	0	0	1	0	656			

Year	Role	Nationality	Team	Ent	Age	Matches	LMatches	Runs	LRuns	...	RunGrowth	MatchGrowth	AverageGrowth	StrikeRateGrowth	WicketGrowth	EconomyChange	AgeSquared	Actual Price	Predicted Price	Absolute Error	
167	2017	Batsman	English	RPS	0	26	12	0	316	0	—	316	12	31.600000	142.980000	12	-7.18	676	145000000	22067188.0	122932812.0
1000	2017	Batsman	English	RCB	0	24	5	0	8	0	—	8	5	2.660000	72.720000	5	-8.57	576	120000000	12767364.0	107232636.0
246	2017	Batsman	South African	KKUP	1	28	5	14	83	161	—	-78	-9	11.580000	-19.150002	0	0.00	784	125000000	25118016.0	96881984.0
530	2017	Wicketkeeper batsman	Indian	RPS	0	36	16	14	290	284	—	6	2	-14.210000	-19.229996	0	0.00	1296	125000000	59694188.0	60305812.0
283	2017	Wicketkeeper	Indian	GL	0	32	14	16	361	335	—	26	-2	10.340000	13.450000	0	0.00	1024	230000000	80662978.0	57862878.0
512	2017	Bowler	Sri Lankan	MI	0	34	12	0	7	0	—	7	12	0.000000	233.330000	11	-8.62	1156	75000000	17865754.0	57304248.0
970	2017	Batsman	Indian	GL	0	31	14	15	442	399	—	43	-1	11.680000	16.090003	1	-0.30	961	125000000	69387048.0	55612952.0
432	2017	All-rounder	Australian	GL	0	27	8	7	54	77	—	-23	1	-12.160000	-35.770000	4	0.03	729	750000000	22074724.0	52925278.0
1092	2017	Batsman	Indian	RCB	0	29	10	16	308	973	—	-665	-6	-50.280002	-29.809999	0	13.00	841	125000000	73324600.0	51675200.0
633	2017	All-rounder	Australian	SRH	0	30	12	17	277	182	—	95	-5	31.000000	21.270000	-11	-2.35	900	100000000	61253960.0	51263960.0
960	2017	Bowler	West Indian	KKR	1	29	16	11	224	7	—	217	5	10.230000	118.460000	-1	0.11	841	950000000	45668100.0	49431900.0
330	2017	Batsman	Indian	KKR	1	36	16	15	468	501	—	-3	1	2.970001	6.130001	0	0.00	1296	125000000	78673088.0	46329912.0
821	2017	Batsman	Indian	MI	0	30	17	14	333	489	—	-156	3	-20.670001	-10.910005	0	0.00	900	125000000	70054128.0	45945872.0
912	2017	Batsman	Indian	SRH	0	32	14	17	479	501	—	-22	-3	-1.689999	10.610001	0	0.00	1024	125000000	81353392.0	43646608.0
887	2017	All-rounder	Australian	RCB	0	36	8	16	71	179	—	-108	-8	-1.930000	-42.560002	-15	-0.55	1296	950000000	53186068.0	41813932.0
662	2017	Wicketkeeper batsman	Indian	SRH	0	34	14	17	79	136	—	-57	-3	6.150000	22.799997	0	0.00	1156	50000000	45745992.0	40745062.0

4.4 Study 2 — Building the Ranking/Classification Pipeline

- Constructed the (player, season) appearance table from the batter, bowler, and non-striker fields.
- Filtered down to the longevity-qualified pool (all 14 seasons, 2012–2025).
- Aggregated delivery-level data to match level, then to career level.
- Computed independent per-metric ranks — average, strike rate, innings, hundreds, fifties for batting; workload and efficiency metrics for bowling.
- Iterated the classification scheme from fixed numeric bands to a final quantile system (§5.3.4).
- Applied a 15-innings minimum-sample threshold before ranking, to exclude small-sample records.

```
Performance matrix saved to player_performance_matrix.csv
```

player	innings	runs	balls_faced	dismissals	not_outs	average	strike_rate	fours	sixes	fifties	hundreds	bowl_innings	balls_bowled	overs	runs_conceded	wickets	economy	bowl_average	bowl_strike_rate
AN Rahane	183	5032	4825	165	18	30.50	125.02	515	123	33	2	1	6	1.0	2	1	2.00	2.00	6.00
B Kumar	71	320	351	38	33	8.42	91.17	31	3	0	0	190	4222	703.7	961	190	1.37	4.85	21.32
DA Miller	133	3077	2230	86	47	35.78	138.60	220	138	13	1	0	0	0.0	0	0	0.00	0.00	0.00
JD Unadkat	27	197	166	16	11	12.31	118.67	16	7	0	0	111	2274	379.0	597	110	1.58	5.43	20.67
MA Agarwal	125	2756	2064	120	5	22.97	133.53	275	100	13	1	0	0	0.0	0	0	0.00	0.00	0.00
NK Pandey	161	3942	3244	135	26	29.20	121.52	340	116	22	1	0	0	0.0	0	0	0.00	0.00	0.00
MS Dhoni	241	5439	3957	142	99	39.30	137.45	375	244	24	0	0	0	0.0	0	0	0.00	0.00	0.00
RA Jadeja	194	3260	2502	117	77	27.86	130.30	240	117	5	0	225	4056	676.0	1135	170	1.68	6.68	23.86
RG Sharma	266	7046	5334	237	29	29.73	132.10	640	303	47	2	32	339	56.5	135	15	2.39	9.00	22.60
SP Narine	118	1780	1069	101	17	17.62	166.51	189	116	7	1	187	4351	725.2	952	192	1.31	4.96	22.66
V Kohli	259	8663	6519	219	40	39.56	132.89	774	292	63	8	26	251	41.0	111	4	2.65	27.75	62.75

```
Ranking (composite z-score) saved to player_ranking.csv
```

player	composite_score	rank
V Kohli	0.577819	1
RG Sharma	0.477852	2
MS Dhoni	0.408890	3
AN Rahane	0.100245	4
DA Miller	0.096304	5
SP Narine	0.031396	6
NK Pandey	-0.017812	7
RA Jadeja	-0.102121	8
MA Agarwal	-0.164795	9
JD Unadkat	-0.841745	10
B Kumar	-1.004435	11

Figure 4.2: Player Ranking and Classification Data Output

4.5 Artifacts Produced

An XGBoost training-and-evaluation script covering all three price-prediction experiments; a match-aggregation and quantile-classification pipeline for the ranking study; the visualizations reproduced in Chapter 8; and this report.

Chapter 5: Technical Details

5.1 Overall Architecture

The two studies are independent pipelines, but they share the same underlying design instinct: prefer rate-based, distribution-relative measures over raw totals or fixed thresholds, and evaluate honestly rather than optimistically. That instinct is what eventually drove both the switch to time-based validation in Study 1 and the switch to quantile classification in Study 2 — they're really the same lesson learned twice.

5.2 Study 1 — XGBoost Auction Price Prediction

5.2.1 Experiment 1 (Baseline)

LabelEncoder for categorical fields, random 80:20 split. Train $R^2 = 0.9888$, Test $R^2 = 0.658$, MAE = ₹12.77M, RMSE = ₹17.72M. The random split accidentally let the model see players from the same years in both training and testing, so it partly memorized typical price ranges instead of generalizing. LabelEncoder also imposed a false ordinal relationship on categorical status codes, which caused the model to systematically underestimate expensive players (₹15M+). Six of the largest errors in this experiment were underpredictions of ₹5–12 Cr on exactly the highest-value players in the test set — see Table 8.1a in §8.2.1 for the actual figures. The XGBoost framework used for the regression model is described in the established XGBoost literature [12].

5.2.2 Experiment 2 (Time-Based Validation) [12]

OneHotEncoder with ColumnTransformer, 7 engineered growth-rate features, time-based split (train 2008–2016, test on 2017 only). Train $R^2 = 0.9888$, Test $R^2 = 0.539$, MAE = ₹14.89M, RMSE = ₹22.36M. The seven engineered features were RunGrowth, MatchGrowth, AverageGrowth, StrikeRateGrowth, WicketGrowth, EconomyChange, and AgeSquared — each built by comparing a player's current-season figure (Runs, Matches, Average, Strike Rate, Wickets, Economy) against their prior-season figure (LRuns, LMatches, etc.), plus a squared age term to let the model capture a non-linear career-age effect.

The drop from 0.658 to 0.539 is not the model getting worse — it's the evaluation getting more honest. IPL auction prices shift year to year (inflation, budget resets, franchise strategy, demand swings), and the 2017 test set contains combinations of price and performance that the model never saw in training. That's exactly the situation a franchise is actually in every year: predicting a price it has never seen before, for a season it has no data from yet.

5.2.3 Experiment 3 (Regularization / Bias-Variance)

Capping tree depth and increasing L1/L2 regularization moved Train R^2 from 0.9888 to 0.7969, and Test R^2 from 0.5395 to 0.4115. [12]

The unregularized model had low bias but high variance — a 0.448 gap between train and test R^2 is a serious overfit. Regularizing narrowed that gap to 0.384, which means variance came down, but test performance dropped too, because the constrained trees could no longer fit patterns that were real and learnable, not just noise. That's the bias-variance tradeoff working exactly as the textbook says it should: the regularized model is safer, but it's also

strictly less capable, and there's no way to regularize your way past a ceiling set by the data itself.

```
=====
MODEL PERFORMANCE
=====
MAE : 17,583,648.00
RMSE : 25,287,235.13
R² : 0.4115
```

Figure 5.1: Regularized XGBoost Model Performance Output

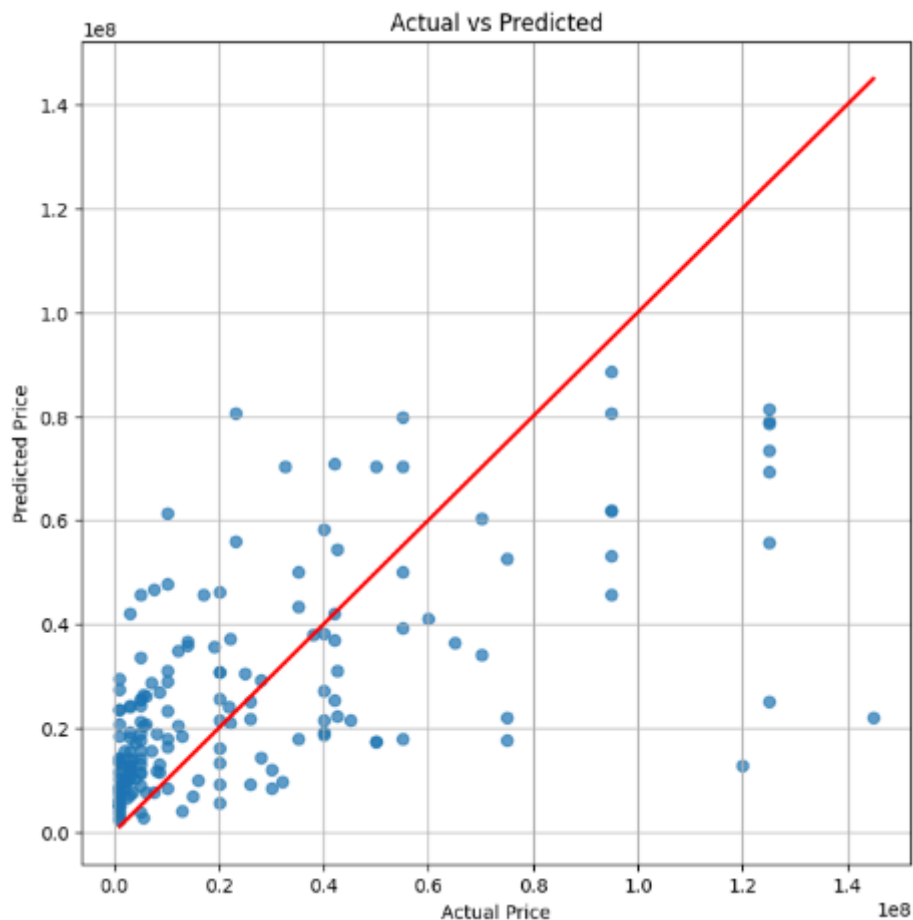


Figure 5.2: Actual vs. Predicted Auction Price

5.2.4 Feature Importance and Leakage

The consistently top-ranked features across all three experiments were Status, LMatches, LRuns, LSixes, LWkts, LFours, Runs, Matches, Age, and Year. Status ranked first every time, and removing it made performance worse — which tells us it carries real signal, but real isn't the same as legitimate. Whether Status counts as leakage depends entirely on whether it reflects something known before bidding starts (like retained-vs-open-pool status, which

would be fine to use) or something only knowable after the auction result (which would be leakage). We flagged this as unresolved rather than guess.

Algorithm 5.1: XGBoost Auction Price Prediction

Input: IPL auction dataset D

1. Load dataset D.
2. Remove irrelevant attributes such as Id, Name and Status.
3. Handle missing numerical and categorical values.
4. Generate performance-growth features using current and previous season statistics.
5. Identify the latest auction year.
6. Split D chronologically:
 - Training set = years before the latest year
 - Testing set = latest year
7. Separate features X from target variable Final Price.
8. Identify numerical and categorical features.
9. Impute numerical features using the median.
10. Impute categorical features using the most frequent value and apply One-Hot Encoding.
11. Configure the XGBoost regression model with controlled tree depth, subsampling, column sampling, and L1/L2 regularization.
12. Combine preprocessing and XGBoost into a pipeline.
13. Train the pipeline using the training dataset.
14. Generate predictions for training and testing data.
15. Calculate MAE, RMSE and R^2 on the testing dataset.
16. Calculate training R^2 and compare it with testing R^2 .
17. Extract and rank feature importance values.
18. Generate actual-vs-predicted, residual, error-distribution and feature-importance visualizations.
19. Calculate absolute prediction errors and identify the largest

errors.

20. Save the trained pipeline for future predictions.

21. Use the trained model to estimate the auction price of a new player profile.

22. Return prediction results, evaluation metrics and feature importance.

	Year	Role	Nationality	Team	Ent	Age	Matches	LMatches	Runs	LRuns	...	RunGrowth	MatchGrowth	AverageGrowth	StrikeRateGrowth	WicketGrowth	EconomyChange	AgeSquared	Actual Price	Predicted Price	Absolute Error	
	167	2017	Batsman	English	RPS	0	26	12	0	316	0	—	316	12	31.600000	142.980000	12	-7.18	676	145000000	22067188.0	122932812.0
	1000	2017	Batsman	English	RCB	0	24	5	0	8	0	—	8	5	2.960000	72.720000	5	-8.57	576	120000000	12767364.0	107232836.0
	246	2017	Batsman	South African	KOP	1	28	5	14	83	161	—	-78	-9	11.560000	-19.150002	0	0.00	784	125000000	25118016.0	99881964.0
	530	2017	Wicketkeeper batsman	Indian	RPS	0	36	16	14	290	284	—	6	2	-14.210000	-19.229996	0	0.00	1296	125000000	55694188.0	69305812.0
	283	2017	Wicketkeeper	Indian	GL	0	32	14	16	361	335	—	26	-2	10.340000	13.450000	0	0.00	1024	230000000	80862976.0	57662976.0
	512	2017	Bowler	Sri Lankan	MI	0	34	12	0	7	0	—	7	12	0.000000	233.330000	11	-8.52	1156	750000000	17665754.0	57304246.0
	970	2017	Batsman	Indian	GL	0	31	14	15	442	399	—	43	-1	11.880000	16.090003	1	-0.30	961	125000000	69387046.0	55912952.0
	432	2017	All-rounder	Australian	GL	0	27	8	7	54	77	—	-23	1	-12.160000	-35.770000	4	0.03	729	750000000	22074724.0	53925276.0
	1052	2017	Batsman	Indian	RCB	0	29	10	16	308	973	—	-65	-6	-50.280002	-29.809999	0	13.00	841	125000000	73324800.0	51975200.0
	633	2017	All-rounder	Australian	SRH	0	30	12	17	277	182	—	95	-5	31.000000	21.270000	-11	-2.35	900	100000000	61253960.0	51253960.0
	960	2017	Bowler	West Indian	KKR	1	29	16	11	224	7	—	217	5	10.230000	118.480000	-1	0.11	841	950000000	45568100.0	49431900.0
	330	2017	Batsman	Indian	KKR	1	36	16	15	498	501	—	-3	1	2.970001	6.130001	0	0.00	1296	125000000	78673088.0	46326912.0
	821	2017	Batsman	Indian	MI	0	30	17	14	333	489	—	-156	3	-20.670001	-10.910005	0	0.00	900	125000000	79054128.0	45945872.0
	912	2017	Batsman	Indian	SRH	0	32	14	17	479	501	—	-22	-3	-1.689999	10.610001	0	0.00	1024	125000000	81353392.0	43646608.0
	887	2017	All-rounder	Australian	RCB	0	36	8	16	71	179	—	-108	-8	-1.930000	-42.560002	-15	-0.55	1296	950000000	53186068.0	41813932.0
	662	2017	Wicketkeeper batsman	Indian	SRH	0	34	14	17	79	136	—	-57	-3	6.150000	22.979997	0	0.00	1156	500000000	45745092.0	40745092.0

Output: Predicted auction prices and model evaluation results

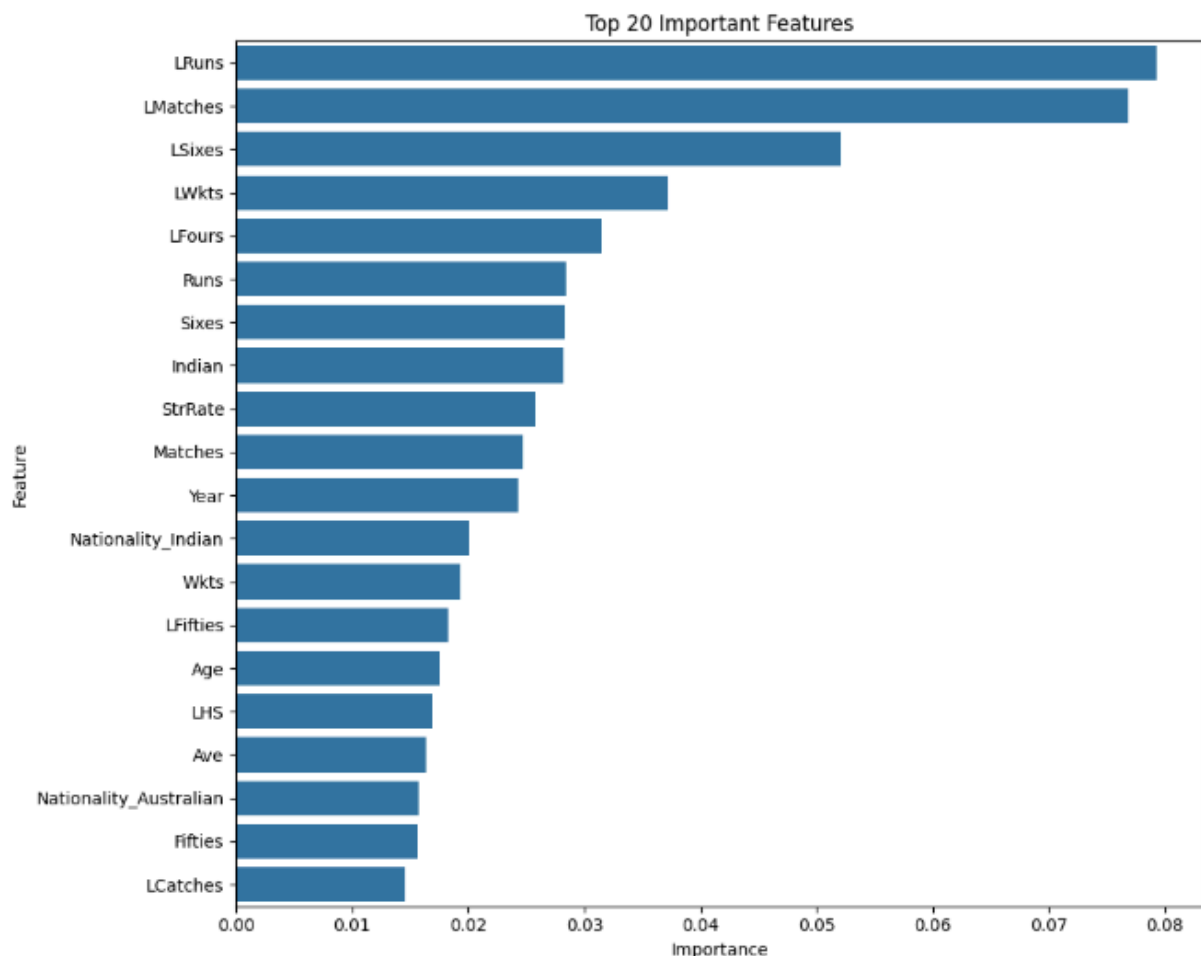


Figure 5.3: Top 20 Features by Importance

5.3 Study 2 — Quantile-Based Player Ranking

5.3.1 Feature Engineering

For each qualifying player, batting figures (innings, runs, balls faced, dismissals, not-outs, average, strike rate, boundaries, milestones) and bowling figures (overs, wickets, economy, bowling average, bowling strike rate) were built by first aggregating delivery-level records to the match level, then summing those match-level totals across the career. Batting average is career runs divided by career dismissals, with an all-not-out fallback of total runs. Strike rate is runs per 100 balls faced. Only valid deliveries — excluding wides and no-balls — counted toward overs bowled, following standard scoring convention.

Performance matrix saved to player_performance_matrix.csv

player	innings	runs	balls_faced	dismissals	not_outs	average	strike_rate	fours	sixes	fifties	hundreds	bowl_innings	balls_bowled	overs	runs_conceded	wickets	economy	bowl_average	bowl_strike_rate
AM Rahane	183	5832	4825	165	18	38.58	125.82	515	123	33	2	1	0	1.0	2	1	2.00	2.00	6.00
S Kumar	71	328	251	38	33	8.42	91.17	31	3	8	0	198	4222	781.7	961	198	1.37	4.85	21.32
DA Miller	133	3877	2220	86	47	35.78	138.68	228	138	13	1	0	0	0.0	0	0	0.00	0.00	0.00
JD Unadkat	27	197	166	16	11	12.31	118.67	36	7	8	0	111	2274	379.8	597	110	1.58	5.43	28.67
MA Agarwal	125	2756	2864	128	5	22.97	133.53	275	188	13	1	0	0	0.0	0	0	0.00	0.00	0.00
RC Pandey	161	3942	3244	135	26	29.28	121.52	348	116	22	1	0	0	0.0	0	0	0.00	0.00	0.00
RS Dhoni	241	5439	3957	142	99	38.38	137.45	375	264	24	0	0	0	0.0	0	0	0.00	0.00	0.00
RA Jadeja	194	3268	2582	117	77	27.86	138.38	248	117	5	0	225	4856	676.8	1135	178	1.68	6.68	23.86
RG Sharma	266	7846	5334	237	29	29.73	132.18	648	383	47	2	32	339	56.5	135	15	2.39	9.00	22.68
SP Narine	118	1788	1869	181	17	17.62	166.51	189	116	7	1	187	4351	725.2	952	192	1.31	4.96	22.66
V Kohli	259	8663	6519	219	48	39.56	132.89	774	292	63	8	26	251	41.8	111	4	2.65	27.75	62.75

Figure 5.4: Player Performance Matrix Used for Ranking

5.3.2 Independent Per-Metric Ranking

Rather than collapsing each player into one blended score right away, every metric was ranked independently across the pool, with rank 1 as best and ties sharing a rank. For batting, average, strike rate, innings, hundreds, and fifties were all ranked higher-is-better. For bowling, innings and overs were ranked higher-is-better as a workload/experience signal, while average and strike rate were ranked lower-is-better as an efficiency signal. Keeping these separate means a player's specific strengths and weaknesses stay visible instead of disappearing into one aggregate number.

An earlier version of this pipeline instead computed a single composite ranking across both roles, using standardized z-scores — each metric centered and scaled to the pool's mean and standard deviation, sign-inverted where lower is better, and combined into a batting-weighted linear sum

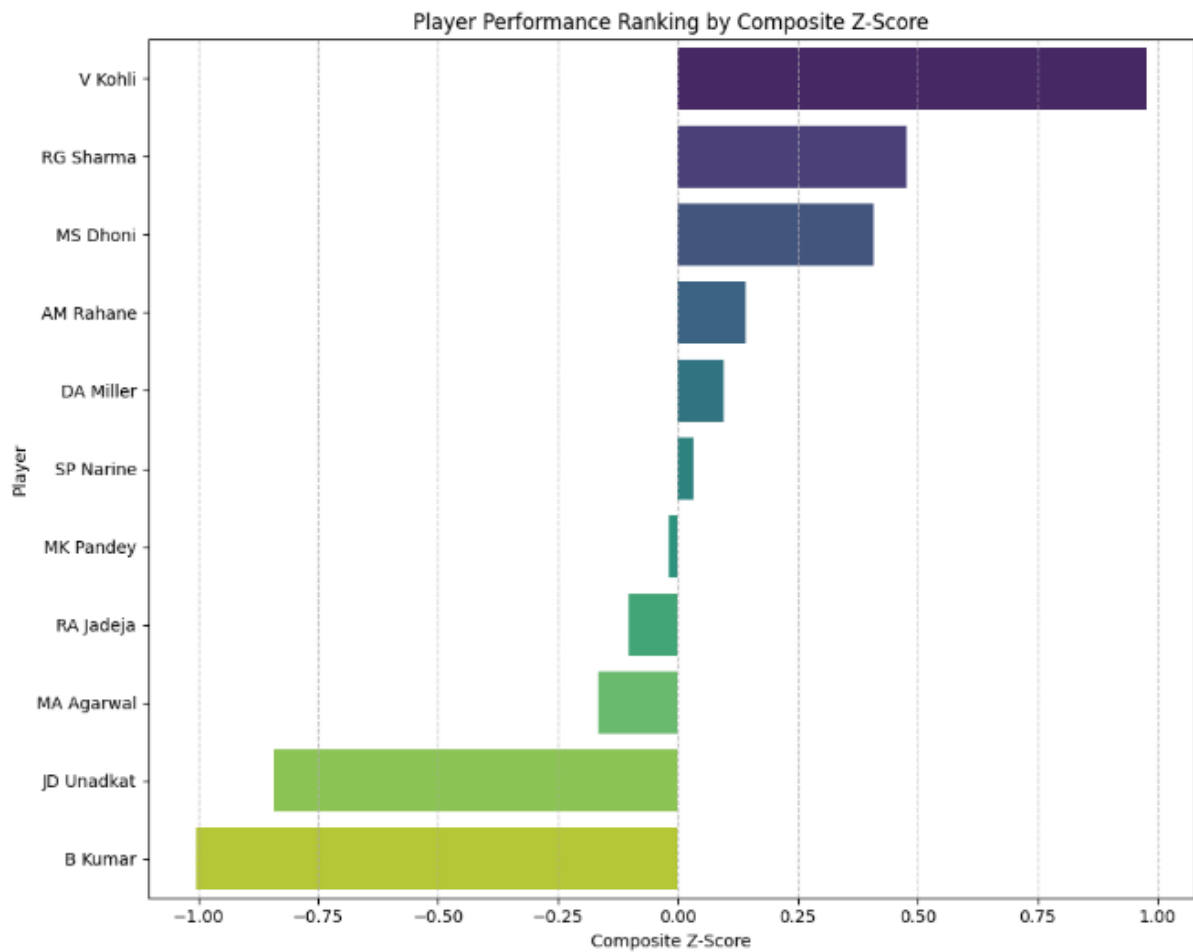


Figure 5.5: Composite Z-Score Player Ranking

This composite view is kept in the report as a supplementary whole-career overview (§8.3.1) but was dropped as the primary output, in favor of the separate batting/bowling tier schemes below — batting and bowling are different enough skills that forcing them onto one shared scale hides more than it reveals (see §6.6 for how we found this out).

5.3.3 Quantile-Based Tier Classification

Batters are split into five tiers by quintiles of the batting-average distribution — bottom 20%, 20–40%, 40–60%, 60–80%, top 20%. Bowlers go into four tiers by quartiles of wickets-per-match ($\text{WPM} = \text{career wickets} \div \text{career bowling innings}$). WPM was chosen over raw wickets or bowling average because it directly captures strike-taking productivity per appearance, independent of how many matches a player happened to feature in.

Quantile boundaries are recomputed from whichever pool is supplied, rather than being fixed numbers. That avoids the degenerate outcomes fixed thresholds produce — an empty tier, or nearly the whole pool landing in one tier — when applied to a different era or a

different-sized player set. Where the pool is too small for a genuine five- or four-way split, a fallback sorted-rank-position procedure is used instead. A minimum-sample threshold of 15 innings was applied before ranking, to keep each classified average or WPM figure representative of a stable career sample rather than a hot streak.

5.3.3.1 Class Definitions

The final classification is distribution-relative. Batters are divided into five quintiles of batting average, while bowlers are divided into four quartiles of wickets-per-match (WPM).

Table 5.1 — Class Definitions

Role	Class	Quantile Range	Definition
Batter	Class 1	Bottom 20%	Lower batting-average group
Batter	Class 2	20–40%	Lower-middle batting-average group
Batter	Class 3	40–60%	Middle batting-average group
Batter	Class 4	60–80%	Upper-middle batting-average group
Batter	Class 5	Top 20%	Highest batting-average group
Bowler	Class 1	Bottom 25%	Lower WPM group
Bowler	Class 2	25–50%	Lower-middle WPM group
Bowler	Class 3	50–75%	Upper-middle WPM group
Bowler	Class 4	Top 25%	Highest WPM group

Batters Ranking and Classification (Quantile-based):											
player	average	strike_rate	innings	hundreds	fifties	avg_rank	sr_rank	innings_rank	hundreds_rank	fifties_rank	Runs_Class
V Kohli	39.58	132.89	259	8	63	1.0	5.0	2.0	1.0	1.0	Class 5 (Avg: Top 20%) (Avg: 39.58)
MS Dhoni	38.30	137.45	241	0	24	2.0	3.0	3.0	8.0	4.0	Class 5 (Avg: Top 20%) (Avg: 38.3)
DA Miller	35.78	138.60	133	1	13	3.0	2.0	7.0	4.0	6.0	Class 4 (Avg: 60-80%) (Avg: 35.78)
AM Rahane	30.50	125.02	183	2	33	4.0	8.0	5.0	2.0	3.0	Class 4 (Avg: 60-80%) (Avg: 30.5)
RG Sharma	29.73	132.10	286	2	47	5.0	6.0	1.0	2.0	2.0	Class 3 (Avg: 40-60%) (Avg: 29.73)
MK Pandey	29.20	121.52	161	1	22	6.0	9.0	6.0	4.0	5.0	Class 3 (Avg: 40-60%) (Avg: 29.2)
RA Jadeja	27.86	130.30	194	0	5	7.0	7.0	4.0	8.0	9.0	Class 2 (Avg: 20-40%) (Avg: 27.86)
MA Agarwal	22.97	133.53	125	1	13	8.0	4.0	8.0	4.0	6.0	Class 2 (Avg: 20-40%) (Avg: 22.97)
SP Narine	17.62	166.51	118	1	7	9.0	1.0	9.0	4.0	8.0	Class 1 (Avg: Bottom 20%) (Avg: 17.62)
JD Unadkat	12.31	118.67	27	0	0	10.0	10.0	11.0	8.0	10.0	Class 1 (Avg: Bottom 20%) (Avg: 12.31)

Figure 5.6: Batting Ranking and Quintile Classification

Bowlers Ranking and Classification (Quantile-based):												
	bowl_innings	overs	wickets	wickets_per_match	economy	bowl_average	bowl_strike_rate	bowl_innings_rank	overs_rank	bowl_avg_rank	bowl_sr_rank	Wickets_Class
player												
B Kumar	190	703.7	198	1.042105	1.37	4.85	21.32	2.0	2.0	1.0	2.0	Class 4 (WPM: Top 25%) (WPM: 1.04)
SP Narine	187	725.2	192	1.026738	1.31	4.96	22.66	3.0	1.0	2.0	4.0	Class 4 (WPM: Top 25%) (WPM: 1.03)
JD Unadkat	111	379.0	110	0.990991	1.58	5.43	20.67	4.0	4.0	3.0	1.0	Class 3 (WPM: 50-75%) (WPM: 0.99)
RA Jadeja	225	676.0	170	0.755556	1.68	6.68	23.86	1.0	3.0	4.0	5.0	Class 2 (WPM: 25-50%) (WPM: 0.76)
RG Sharma	32	56.5	15	0.468750	2.39	9.00	22.60	5.0	5.0	5.0	3.0	Class 1 (WPM: Bottom 25%) (WPM: 0.47)
V Kohli	26	41.8	4	0.153846	2.65	27.75	62.75	6.0	6.0	6.0	6.0	Class 1 (WPM: Bottom 25%) (WPM: 0.15)

Figure 5.7: Bowling Ranking and Quartile Classification

5.3.4 Development Iteration

This methodology went through ten documented refinement steps rather than arriving fully formed:

- 1. Initial fixed-band classification on total runs and wickets.
- 2. Confirmed aggregation needed to happen at match level, not raw delivery level.
- 3. Switched the driving statistic from career totals to per-career averages.
- 4. Defined fixed numeric bands on those averages.
- 5. Revised tier labels to show each player's own numeric value alongside their tier, not just the tier name.
- 6. Changed the bowling driver from bowling average to wickets-per-match.
- 7. Replaced the fixed bands with the quantile scheme.
- 8. Fixed an index/column lookup defect in the tier-label mapping (detailed in §6.7).
- 9. Reverted a short-lived numeric-only tier field back to the labeled version.
- 10. Added the 15-innings minimum-sample safeguard.

The overall progression — totals, to absolute rates, to relative quantile-normalized rates — reflects increasing robustness to playing-time imbalance and to the size of whatever pool the method is eventually run on.

Algorithm 5.2: Quantile-Based Player Ranking and Classification

Input: IPL ball-by-ball delivery dataset.

1. BEGIN
2. Load the IPL ball-by-ball dataset.
3. Identify the seasons covered by the dataset.
4. Create a list of players who participated in every required IPL season within the selected period.
5. Filter the dataset to retain records involving the qualifying players as batter, bowler, or non-striker.
6. FOR each qualifying player (P) DO:
7. Calculate batting statistics:
 - a. Identify matches in which P was the batter.
 - b. Calculate innings played.
 - c. Calculate total runs.
 - d. Calculate balls faced.
 - e. Calculate dismissals and not-outs.
 - f. Calculate batting average.
 - g. Calculate strike rate.
 - h. Count fours and sixes.
 - i. Count fifties and hundreds.
8. Calculate bowling statistics:
 - a. Identify matches in which P was the bowler.
 - b. Calculate bowling innings.
 - c. Calculate valid balls bowled.
 - d. Convert balls into overs.
 - e. Calculate runs conceded.
 - f. Calculate total wickets.
 - g. Calculate economy rate.
 - h. Calculate bowling average.
 - i. Calculate bowling strike rate.
9. Store all calculated batting and bowling statistics in the player performance matrix.
10. END FOR

11. Define performance metrics where a higher value indicates better performance:
Runs, Average, Strike Rate, Fours, Sixes, Fifties, Hundreds and Wickets.
12. Define performance metrics where a lower value indicates better performance:
Economy Rate, Bowling Average and Bowling Strike Rate.
13. Calculate the Z-score for every performance metric:
$$Z = (X - \text{Mean}) / \text{Standard Deviation}$$
14. For metrics where lower values indicate better performance, reverse the Z-score direction.
15. Assign weights to the standardized metrics according to their contribution to the overall performance.
16. Calculate the Composite Z-Score:
$$\text{Composite Score} = \Sigma (\text{Weighted Z-score of each metric})$$
17. Rank players in descending order of Composite Score.
18. Assign Rank 1 to the player with the highest Composite Score.
19. Sort the player ranking according to the assigned rank.
20. Save the performance matrix containing the raw performance statistics.
21. Save the ranking dataset containing Z-scores, Composite Scores and player ranks.
22. Generate visualizations of player performance and Composite Z-Score rankings.
23. RETURN:
Player Performance Matrix,
Z-scores,
Composite Scores,
Player Rankings
24. END

Output: Batting classes, bowling classes, rankings and comparison outputs.

5.3.5 Pairwise Head-to-Head Comparison

The tier classification in §5.3.3 answers "where does this player sit relative to the pool," but Prof. Mehta pointed out during a review that it doesn't actually answer the question a franchise asks most often at an auction table, which is simpler and more direct: how does this specific player compare to that specific player, right now, on the metrics that matter. Two players can land in the same tier and still be quite different picks depending on who else is already in the squad, so we built a second, complementary layer on top of the tier system rather than replacing it.

For every pair of players in a role group, we compute the difference in a given metric between the row player and the column player. For batting metrics — average, strike rate — a positive number in a cell means the row player is ahead. For bowling metrics, where a lower number is actually better (you want a low average and a low economy, not a high one), we flip the sign before plotting, so that green still means "row player wins" and red still means "column player wins" no matter which metric is being shown. Without that flip, a bowling heatmap would show green for players who are actually worse, which defeats the point of a visual comparison tool.

Each pair is computed once and mirrored, so the matrix is antisymmetric by construction — if Kohli beats Dhoni by 1.26 on average, Dhoni loses to Kohli by exactly -1.26 , and the diagonal (a player compared to himself) is always zero. That's a useful sanity check in itself: if the diagonal isn't zero somewhere, the pipeline has a bug.

We built four of these matrices in total — batting average, batting strike rate, bowling average, and economy rate — one heatmap grid per role, per metric, rendered with a red-to-green diverging colormap centered on zero so the eye is drawn straight to the extremes rather than having to read every number.

Heatmap Color Interpretation

The head-to-head heatmaps use a red-to-green diverging color scale centered at zero to visually represent pairwise performance differences. **Green indicates that the row player performs better than the column player**, while **red indicates that the column player performs better than the row player**. Yellow represents values close to zero, indicating relatively similar performance. The intensity of each color reflects the magnitude of the performance difference, with darker shades indicating larger differences. For bowling average and economy rate, the differences are sign-adjusted because lower values indicate better performance. This ensures that the color interpretation remains consistent across all four heatmaps. The diagonal cells are always zero, as each player is compared with themselves.



Figure 5.8: Batting Average Head-to-Head Comparison

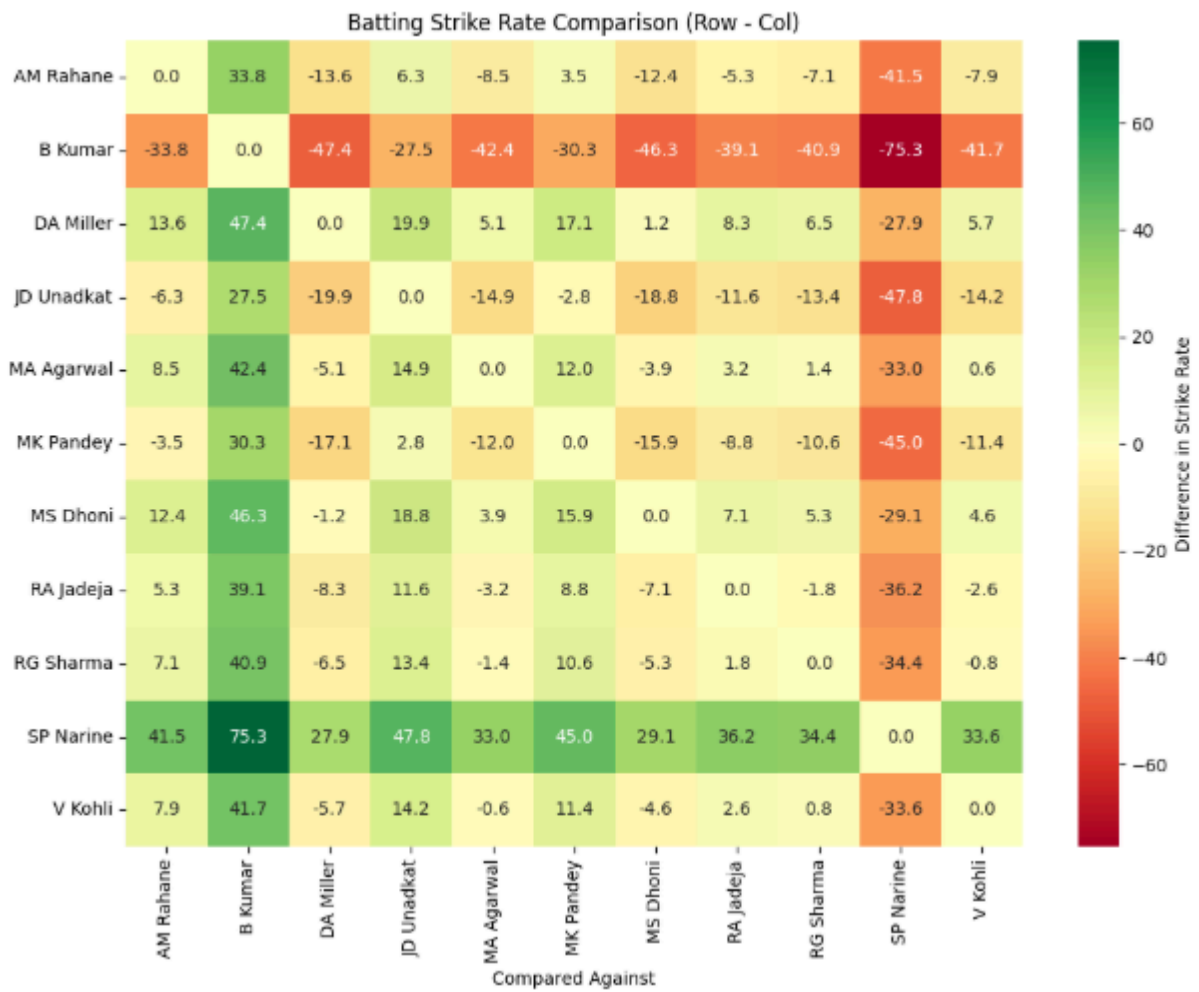


Figure 5.9: Batting Strike Rate Head-to-Head Comparison

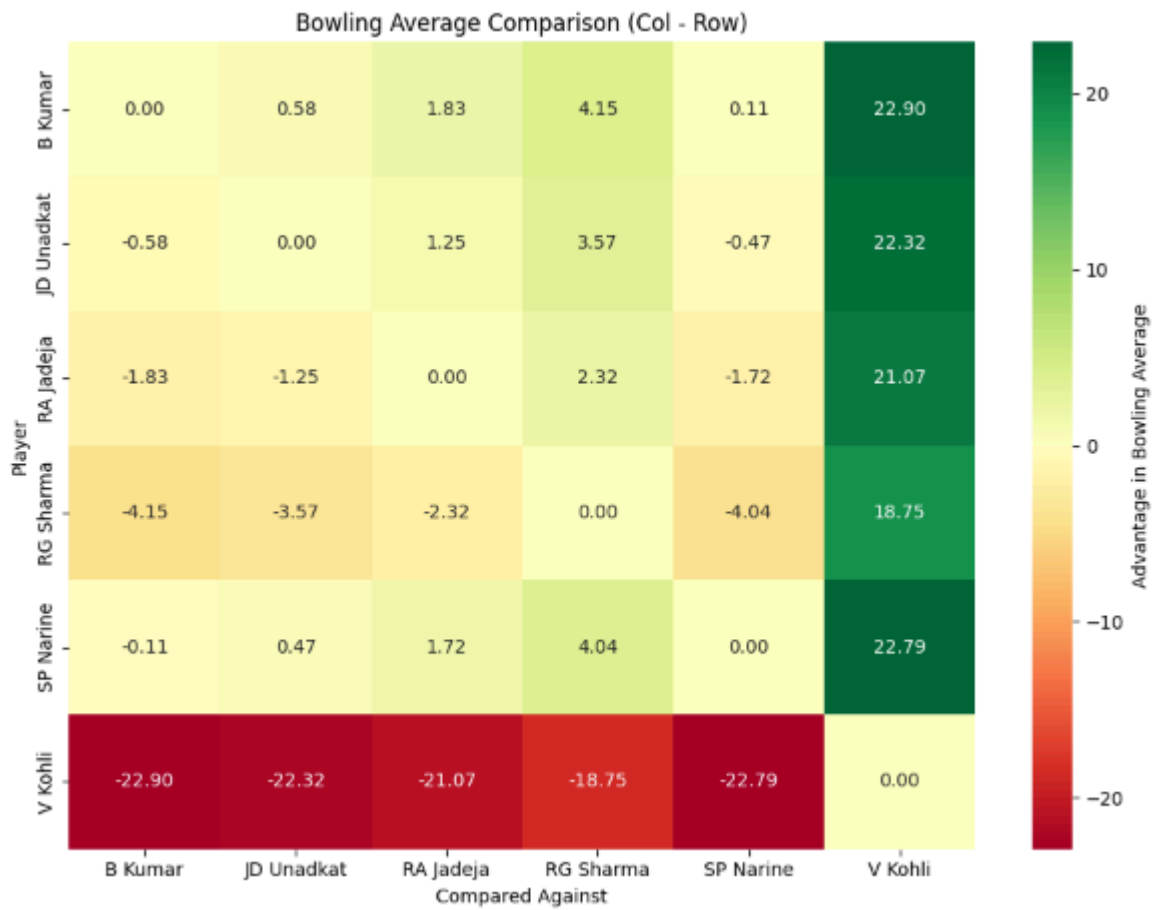


Figure 5.10: Bowling Average Head-to-Head Comparison



Figure 5.11: Economy Rate Head-to-Head Comparison

Chapter 6: Challenges and How We Solved Them

This chapter goes through the specific problems we ran into while building both studies, in the order we hit them. Chapters 4 and 5 mention some of these in passing; this chapter is where we actually walk through what broke, how we noticed, what we tried, what fixed it, and what it cost us. We think this is the most honest part of the report, because none of it could have been written without actually running the code.

6.1 Ball-by-ball double-counting in career totals

What broke: our first pass at career statistics summed runs and balls directly from the raw ball-by-ball delivery table. The totals came out far too high — well beyond what's plausible for any real player's IPL career.

How we noticed: we spot-checked a handful of well-known players against publicly known career figures, and the numbers we'd computed were inflated, in some cases by a wide margin.

What we hypothesized: a single IPL match can generate multiple delivery-level rows referencing the same player in different capacities — as striker, non-striker, or bowler across different balls — and summing raw rows was letting some contributions get counted more than once within a single match.

What fixed it: we restructured the pipeline to aggregate delivery-level records to the match level first, then summed those match-level totals across the career, instead of summing raw delivery rows directly. That two-step aggregation is now standard across both studies (§5.3.1).

What it cost: this meant redoing the entire career-statistics computation and re-validating every downstream table and chart in Study 2 against the corrected numbers. It also meant we couldn't fully trust anything computed before this fix, which is part of why the design history in §5.3.4 has as many steps as it does — several early iterations were built on the uncorrected numbers.

6.2 LabelEncoder's false ordinality on the Status field

What broke: our baseline XGBoost model (Experiment 1) systematically underpredicted prices for the most expensive players.

How we noticed: this showed up while sanity-checking predictions against actual sale prices for the highest-value players in the dataset — the gap between predicted and actual price was consistently large and consistently in the same direction.

What we hypothesized: LabelEncoder assigns arbitrary integer codes to categorical values, and a tree-based model can end up treating those integers as if they carried ordinal meaning — as if 'category 3' were meaningfully greater than 'category 1' in a way that has nothing to do with the actual categories.

What we tried and what fixed it: we replaced LabelEncoder with a OneHotEncoder inside a ColumnTransformer for all categorical fields, removing any implied ordering, in Experiment 2.

What it cost: fixing this required rebuilding the entire preprocessing pipeline, not just swapping one line, since OneHotEncoder changes the shape of the feature matrix that everything downstream depends on.

6.3 Random-split evaluation was quietly optimistic

What broke: nothing broke, exactly — Experiment 1's test R^2 of 0.658 looked like a solid result on its face.

How we noticed: we started questioning it once we thought carefully about what a random 80:20 split actually does to a dataset with a time dimension. A random split can put a player's 2015 auction record in training and a similar player's 2015 record in testing, meaning the model gets to see the general price landscape of a year it's supposedly being tested on.

What we tried: we switched to a time-based split — training on 2008–2016 and testing only on the held-out 2017 season, which the model never saw during training in any form.

What fixed it / what it cost: test R^2 dropped to 0.539. We don't think of this as the model getting worse — it's the same model being evaluated honestly for the first time. The cost is that 0.539 is a less impressive number to report, but it's the number that actually predicts how the model would perform on a genuinely unseen future season, which is the only situation that matters in practice.

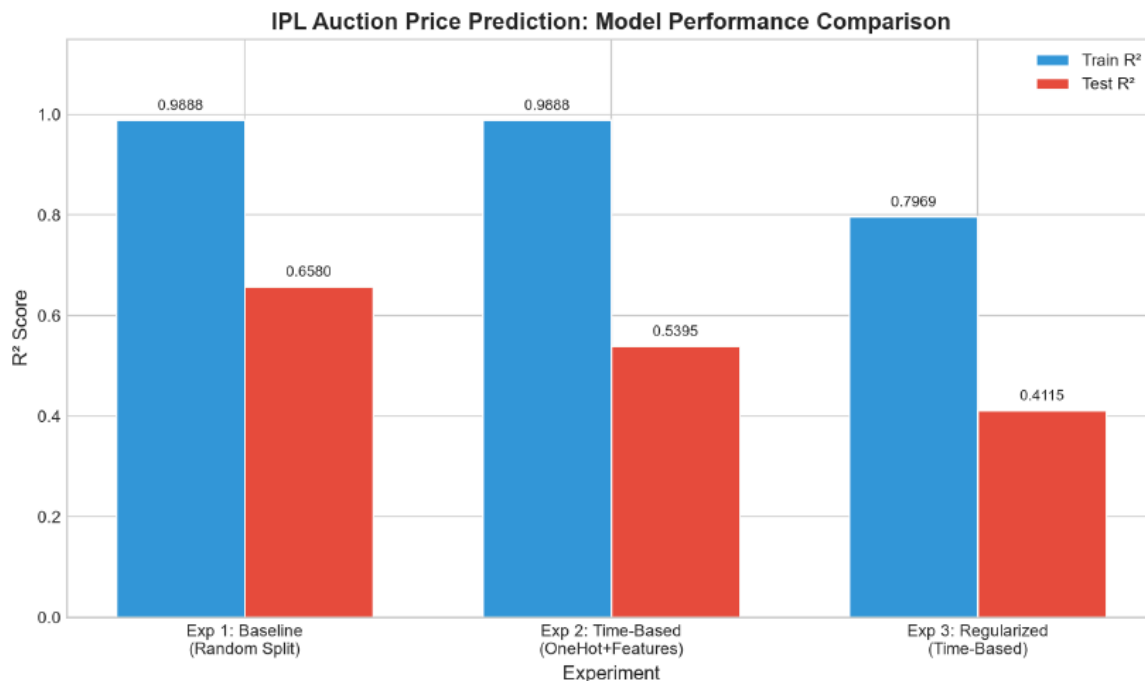


Figure 6.1: Model Performance Comparison Across Experiments

6.4 Regularizing to fix overfitting made test performance worse, not better

What broke: after switching to the time-based split, the train-test R² gap in Experiment 2 was still large (0.9888 vs 0.5395), which is a classic overfitting signature. Our first instinct was that adding regularization would close that gap and also improve test performance.

What we tried: we capped tree depth and added L1/L2 regularization in Experiment 3.

What happened instead: the train-test gap did narrow, from 0.448 to 0.384, confirming reduced overfitting. But test R² also fell further, to 0.4115.

How we made sense of it: this is the bias-variance tradeoff, not a failed fix. Constraining the trees reduced variance, but it also increased bias enough that the model lost the ability to fit patterns in the data that were real, not noise. We realized the actual bottleneck wasn't model complexity — it was that the dataset simply doesn't contain enough of the right information (retention history, recent form, franchise budget signals) for any model, regularized or not, to do much better than this on the given features.

What it cost: this was the point where we accepted that further tuning wasn't going to meaningfully move the number, and redirected effort toward documenting the ceiling honestly rather than chasing a better R² that the data wasn't going to support.

6.5 Only four players qualified for the full 2008–2025 window

What broke: our original plan was to build the longevity-qualified player pool across the entire available range, 2008–2025.

How we noticed: when we ran the season-by-season appearance filter across that full range, only four players had appeared in literally every season.

What we tried: four players isn't enough to build any kind of meaningful quantile classification — there's no distribution to speak of.

What fixed it: we relaxed the window to 2012–2025 (14 seasons instead of 18), which brought the qualifying pool up to 11 players — still small, but workable.

What it cost: this means the ranking study's conclusions are scoped to players active across a shorter, more recent window, and it's part of why we flag the small-pool limitation directly in §5.3.3 rather than let a reader assume the classification generalizes to a larger set of players than it actually does.

6.6 The composite z-score buried specialist bowlers

What broke: our first version of the ranking output used a single composite z-score combining batting and bowling metrics into one number per player.

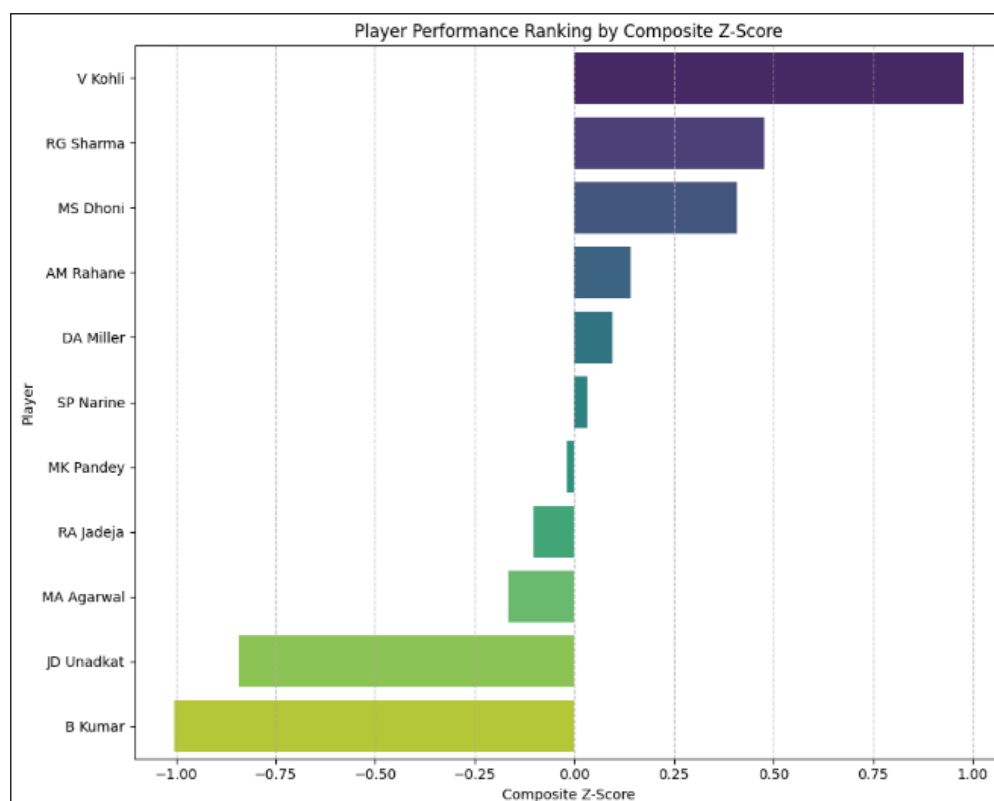


Figure 6.2: Composite Z-Score Ranking — Diagnostic View

How we noticed: specialist bowlers like B Kumar and S P Narine — both clearly elite at their actual job — came out ranked at the bottom of the composite list, below several players who were, by any reasonable cricketing judgment, less accomplished overall.

What we hypothesized: the composite score was weighted toward batting metrics (since the qualifying pool skews toward specialist batters), so a player whose entire contribution is bowling had almost nothing in the score that reflected their actual skill.

What fixed it: rather than trying to reweight the composite score to fix this, we abandoned it as the primary output entirely and moved to separate batting and bowling classification tracks (§5.3.2, §5.3.3), so a specialist bowler is judged against other bowlers, not folded into a batting-dominated scale.

What it cost: we kept the composite score in the report as a supplementary view (§8.3.1) because it's a useful illustration of exactly this failure mode, but we no longer treat it as a real ranking of player value.

6.7 A tier-label lookup defect

What broke: after switching to the quantile classification scheme, some players' displayed tier labels didn't match the numeric value sitting right next to them in the same row.

How we noticed: manually checking a few rows against their raw averages during a routine sanity pass — the mismatch was obvious once we looked, but easy to miss at a glance.

What we hypothesized and found: the tier-label mapping was reading a player's identity from a dataframe column instead of from the table's index, so labels were occasionally being matched to the wrong row after a sort or a filter operation shifted the row order.

Batters Ranking and Classification (Quantile-based):											
	average	strike_rate	innings	hundreds	fifties	avg_rank	sr_rank	innings_rank	hundreds_rank	fifties_rank	Runs_Class
player											
V Kohli	39.56	132.89	259	8	63	1.0	5.0	2.0	1.0	1.0	Class 5 (Avg: Top 20%) (Avg: 39.56)
MS Dhoni	38.30	137.45	241	0	24	2.0	3.0	3.0	8.0	4.0	Class 5 (Avg: Top 20%) (Avg: 38.3)
DA Miller	35.78	138.60	133	1	13	3.0	2.0	7.0	4.0	6.0	Class 4 (Avg: 60-80%) (Avg: 35.78)
AM Rahane	30.50	125.02	183	2	33	4.0	8.0	5.0	2.0	3.0	Class 4 (Avg: 60-80%) (Avg: 30.5)
RG Sharma	29.73	132.10	266	2	47	5.0	6.0	1.0	2.0	2.0	Class 3 (Avg: 40-60%) (Avg: 29.73)
MK Pandey	29.20	121.52	161	1	22	6.0	9.0	6.0	4.0	5.0	Class 3 (Avg: 40-60%) (Avg: 29.2)
RA Jadeja	27.86	130.30	194	0	5	7.0	7.0	4.0	8.0	9.0	Class 2 (Avg: 20-40%) (Avg: 27.86)
MA Agarwal	22.97	133.53	125	1	13	8.0	4.0	8.0	4.0	6.0	Class 2 (Avg: 20-40%) (Avg: 22.97)
SP Narine	17.62	166.51	118	1	7	9.0	1.0	9.0	4.0	8.0	Class 1 (Avg: Bottom 20%) (Avg: 17.62)
JD Unadkat	12.31	118.67	27	0	0	10.0	10.0	11.0	8.0	10.0	Class 1 (Avg: Bottom 20%) (Avg: 12.31)

Figure 6.3: Corrected Batting Tier Classification Output

What fixed it: we corrected the mapping to read tier labels directly from the index rather than from a column, and re-verified every row afterward.

What it cost: mainly time — this kind of bug doesn't throw an error, it just silently produces wrong output, so catching it required going back and manually re-checking results we'd already treated as finished.

6.8 Auction dataset gaps that no amount of modeling could fix

What broke: nothing we could fix in code. The 2008–2017 auction records don't include retention history, franchise budget signals, injury status, or recent-form indicators.

How we noticed: these fields simply aren't present in the dataset, and their absence became more obvious the more we tried to explain remaining prediction error through feature engineering and tuning, without much success.

What we tried: additional feature engineering (the growth-rate features in §5.2.2) and the regularization pass in §6.4 were both partly attempts to squeeze more signal out of the data available.

What we concluded: the ceiling on Study 1's performance is set by what the dataset contains, not by model choice. We documented this explicitly rather than continuing to tune a model that had already extracted most of what was extractable from the available features.

What it cost: we can't claim the price-prediction model as a tool a franchise could use for real bidding decisions. We frame it instead as a diagnostic tool — useful for flagging when a price looks structurally out of line with a player's stats, not for setting an actual bid.

Chapter 7: Learning Outcomes

7.1 Technical Skills

Match-level data aggregation to avoid double-counting in ball-by-ball data; the practical difference between LabelEncoder and OneHotEncoder on categorical sports data, and why the choice matters for tree-based models specifically; why time-based train/test splitting matters for any dataset with a temporal dimension, and how misleading a random split can look before you think about it carefully; reading the bias-variance tradeoff directly off a train/test R^2 gap instead of just chasing a lower training error; feature-importance-based leakage diagnosis; and designing a distribution-relative classification scheme instead of relying on fixed thresholds that don't generalize across different data pools.

7.2 Professional Skills

Breaking an open-ended research problem into concrete, checkable weekly objectives; documenting a design's full iteration history rather than only its final state, so the reasoning behind it is auditable later (Chapter 6 is really the product of this habit); collaborating on a shared literature review while each of us owned an individual technical deliverable; and writing findings up as a report that's honest about limitations rather than one that only shows the best numbers.

Chapter 8: Results and Discussion

8.1 Overview

This chapter presents results from both studies: the XGBoost auction price prediction model (Study 1) and the quantile-based player ranking and classification system (Study 2). Results are presented with the supporting visualizations, followed by a discussion of what they reveal about data-driven player valuation in the IPL context.

8.2 Study 1 Results: Auction Price Prediction

8.2.1 Benchmarking and Experiment Comparison

The three experiments were benchmarked using training R^2 , testing R^2 , MAE, RMSE and the train-test R^2 gap. The time-based evaluation is particularly important because the 2017 season is held out as an unseen future season.

The random-split result provides an optimistic baseline, while the time-based experiment provides the more realistic generalization estimate. The regularized experiment is useful for diagnosing the bias-variance tradeoff rather than simply maximizing test R^2 .

Table 8.1 — Experiment comparison / benchmarking

Experiment	Split	Train R^2	Test R^2	MAE	RMSE
Exp 1: Baseline	Random 80:20	0.9888	0.6580	₹12.77 M	₹17.72 M
Exp 2: Time-based, improved preprocessing	Time-based (2017 held out)	0.9888	0.5395	₹14.89 M	₹22.36 M
Exp 3: Regularized	Time-based (2017 held out)	0.7969	0.4115	—	—

Table 8.2 — Largest prediction errors, Experiment 1

Role	Team	Actual Price	Predicted Price	Absolute Error
Batsman	RPS	₹1,45,00,000	₹22,06,718.8	₹1,22,93,281.2
Batsman	RCB	₹1,20,00,000	₹12,76,736.4	₹1,07,23,263.6
Batsman	KXIP	₹1,25,00,000	₹25,11,801.6	₹99,88,198.4

Role	Team	Actual Price	Predicted Price	Absolute Error
Bowler	MI	₹75,00,000	₹1,78,95,754.0	₹57,30,424.6
Batsman	GL	₹1,25,00,000	₹69,38,708.0	₹55,61,292.0
All-rounder	GL	₹75,00,000	₹22,07,472.4	₹52,92,527.6

Every one of these is the model landing far below the actual sale price for a player who ultimately sold high — not a scattering of random misses, but a consistent direction of error concentrated at the top of the price range. This is the concrete evidence behind the underestimation claim in §5.2.1.

The gap between Experiment 1 and Experiment 2's test R^2 (0.658 down to 0.539) is the central result of this study — not because the model got worse, but because the random split had been quietly leaking information from the test years into training. The near-identical train R^2 (0.9888) across both experiments shows the model memorized the training set to almost the same degree in both cases; the difference is entirely in how honestly each split measured generalization to unseen data.

8.2.2 Bias-Variance Effect of Regularization

8.2.2.1 Practical Significance of Variance and Parameters

Variance

In the IPL auction-price prediction study, model variance represents the sensitivity of the model to the training data. A large difference between training and testing R^2 indicates that the model may be fitting patterns specific to historical observations rather than relationships that generalize to an unseen auction season. Reducing excessive variance is therefore important for obtaining more stable predictions on new auction data.

Tree Depth

Tree depth controls the complexity of the XGBoost decision trees. Greater depth allows the model to capture more complex relationships among player statistics, but excessive depth can increase overfitting and therefore increase variance.

L1/L2 Regularization

L1 and L2 regularization constrain model complexity. In Experiment 3, regularization reduced the train-test R^2 gap from 0.448 to 0.384, indicating reduced variance, while test R^2 also decreased. This demonstrates the practical bias-variance tradeoff.

R^2

R^2 indicates the proportion of variation in auction price explained by the model under the selected evaluation setup. The time-based R^2 is more informative for future-season generalization because the 2017 season was held out from training.

Feature Parameters

Variables such as Status, previous matches, previous runs, previous wickets, age and year provide measurable information for prediction. However, the dataset does not include several real-world auction factors such as retention history, franchise budget, injury status and recent-form information. Therefore, model performance is constrained by both model design and the information available in the dataset.

Regularization in Experiment 3 narrowed the train-test R^2 gap from 0.448 (unregularized) to 0.384, confirming reduced overfitting — but test R^2 also fell, from 0.5395 to 0.4115. Constraining the trees reduced variance but increased bias enough that some genuine, non-noisy patterns in the data were no longer captured. In practical terms, Experiment 2 — unregularized, time-based — is the more useful of the three models: Experiment 1 is optimistic in a way that would mislead a franchise, and Experiment 3 is safer but strictly less accurate on data it hasn't seen.

8.2.2.2 Dataset Limitation and Transition to Player Performance Analysis

The three auction-price prediction experiments provided useful information about the predictive behaviour of the XGBoost model and the effect of temporal validation and regularization. However, the experiments also highlighted a limitation of the auction dataset. The dataset primarily contained **season-level player statistics and auction-related attributes**, such as matches, runs, batting average, strike rate, wickets, economy rate, previous-season statistics, player attributes and final auction price. Although these variables were sufficient for developing an auction-price prediction model, they provided limited granularity for analysing how a player's performance was accumulated at the individual match and delivery level.

This limitation became particularly important when the objective was extended from **predicting auction price** to **comparing and ranking players based on sustained performance**. A more detailed dataset was therefore required to capture the underlying events contributing to player statistics rather than relying only on aggregated seasonal values.

To address this limitation, a **ball-by-ball IPL dataset** was used for the second study. The dataset contains individual delivery-level records, allowing player performance to be

reconstructed through aggregation across matches and seasons. The analysis first identified players meeting the required longevity criterion and then calculated detailed batting and bowling statistics from their match-level and delivery-level records. The resulting performance matrix included measures such as innings, runs, balls faced, dismissals, batting average, strike rate, fours, sixes, fifties, hundreds, bowling innings, balls bowled, overs, runs conceded, wickets, economy rate, bowling average and bowling strike rate.

This transition changed the objective of the analysis from **auction-price prediction** to **multi-dimensional player performance evaluation**. Since the performance metrics have different scales and different interpretations, direct comparison using raw values would not provide a consistent overall ranking. Therefore, the study standardized the selected performance metrics using **Z-scores** and combined them using weighted composite scoring. Metrics where higher values represented stronger performance were assigned positive Z-scores, while metrics where lower values were preferable, such as economy rate and bowling average, were direction-adjusted.

The resulting **Composite Z-Score** provided a standardized basis for comparing players across multiple performance dimensions. Players were then ranked according to their composite scores, providing the foundation for the subsequent **quantile-based classification of batting and bowling performance**.

8.2.3 Feature Importance and Leakage

The top predictive features — Status, LMatches, LRuns, LSixes, LWkts, LFours, Runs, Matches, Age, and Year — stayed stable across all three experiments. Status ranked first every time, and removing it made performance worse, which is informative but not conclusive on its own: Status is a legitimate predictor only if it reflects something known before bidding starts, and it's leakage if it's contaminated by the auction outcome itself. As noted in §5.2.4, this is left as an open question pending a closer look at what the field actually encodes.

Top 20 Features		
	Feature	Importance
6	LRuns	0.079341
4	LMatches	0.076809
20	LSixes	0.052062
26	LWkts	0.037176
18	LFours	0.031470
5	Runs	0.028452
19	Sixes	0.028288
33	Indian	0.028201
11	StrRate	0.025806
3	Matches	0.024678
0	Year	0.024316
61	Nationality_Indian	0.020015
25	Wkts	0.019343
14	LFifties	0.018267
2	Age	0.017531
8	LHS	0.016872
9	Ave	0.016302
57	Nationality_Australian	0.015763
13	Fifties	0.015643
22	LCatches	0.014576

Figure 8.1: Feature Importance and Leakage Analysis

8.3 Study 2 Results: Player Ranking and Classification

8.3.1 Composite Z-Score Ranking (Supplementary)

The supplementary composite ranking places V Kohli first by a wide margin (composite score 0.977), followed by RG Sharma (0.482) and MS Dhoni (0.409). AM Rahane and DA Miller sit in the middle tier (0.141 and 0.095), while SP Narine and MK Pandey land close to zero (0.030 and -0.018) — reflecting that their contribution is bowling-heavy in a scoring scheme weighted toward batting. B Kumar (-1.005) and JD Unadkat (-0.843) rank lowest, which says nothing about their bowling quality — both are elite strike bowlers whose value simply doesn't register on a batting-weighted scale. This is exactly the failure mode described in §6.6, and it's why the composite score isn't used as the primary output anywhere else in this report.

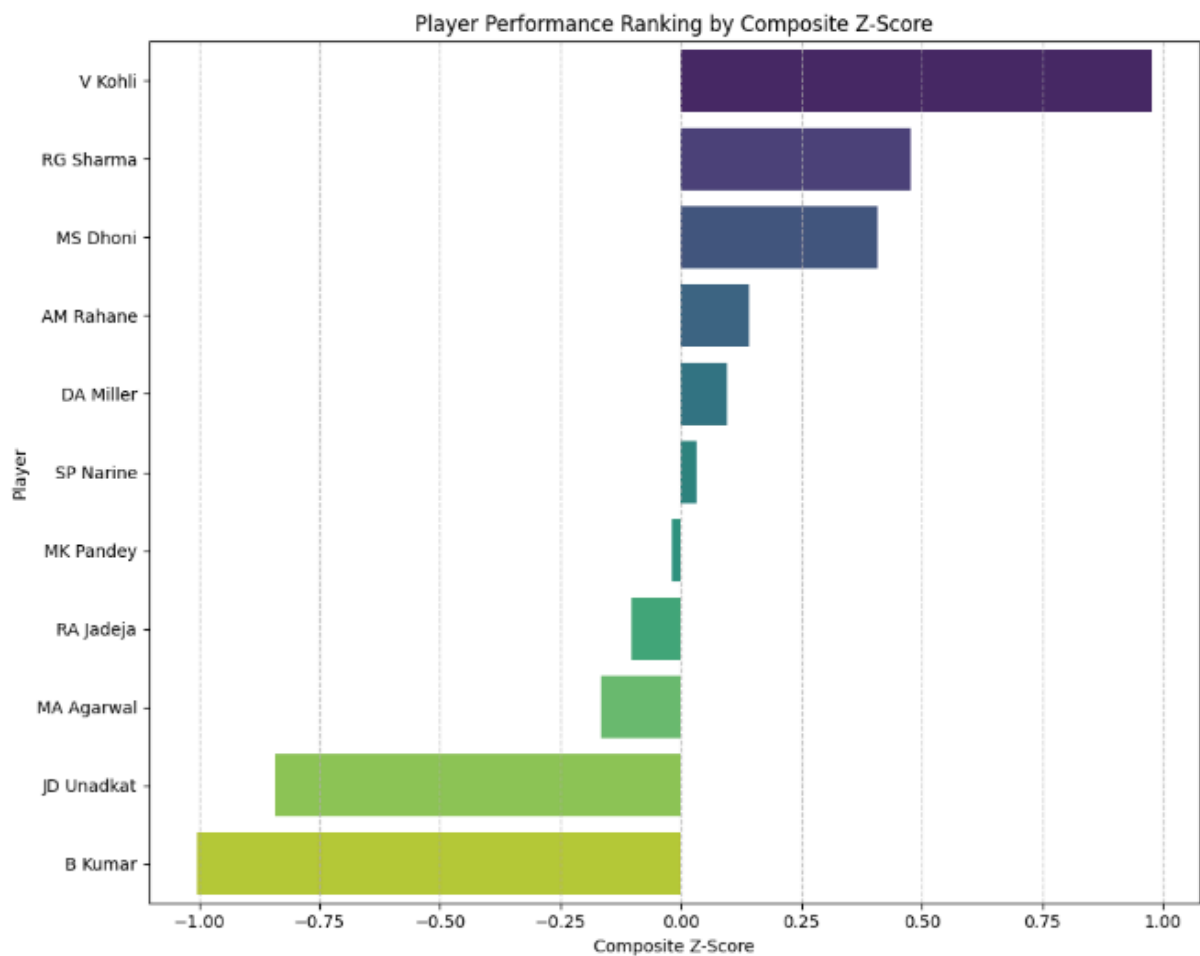


Figure 8.2: Composite Z-Score Ranking (Supplementary)

8.3.2 Batting Performance Comparison

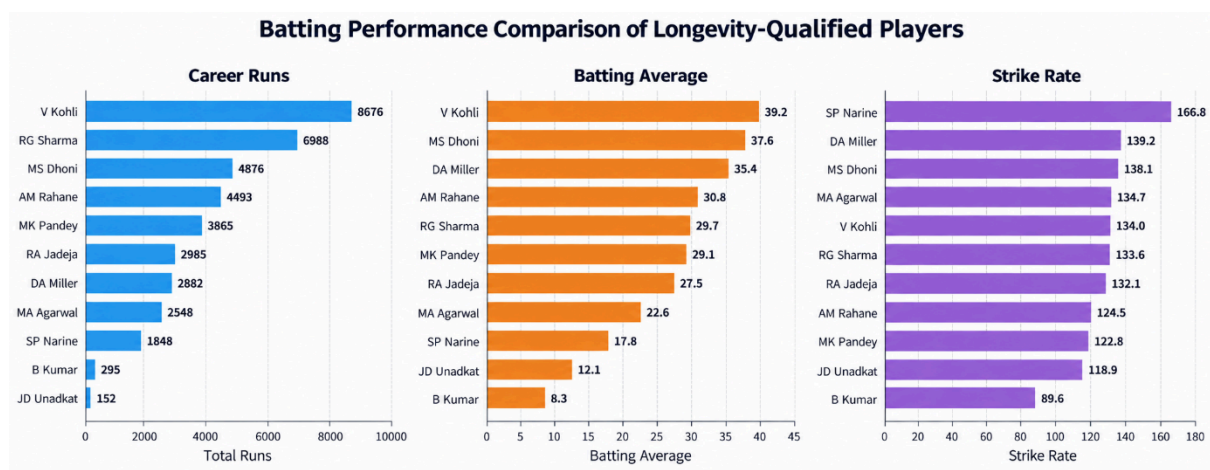


Figure 8.3: Batting Performance Comparison

Career runs are led by V Kohli, RG Sharma, and MS Dhoni. On batting average, the ordering shifts: Kohli and Dhoni lead, but DA Miller moves ahead of the higher-run-scoring Rahane, Sharma, and Pandey, because Miller's fewer innings carry a higher yield per dismissal. Strike

rate reorders the pool again — SP Narine is the clear outlier leader, reflecting his role as a late-innings hitter across a smaller sample of high-tempo innings, ahead of Miller and Dhoni. No single player leads on all three axes at once, which is the direct, empirical reason we rank each metric independently (§5.3.2) instead of collapsing batting quality into one number.

8.3.3 Bowling Performance Comparison

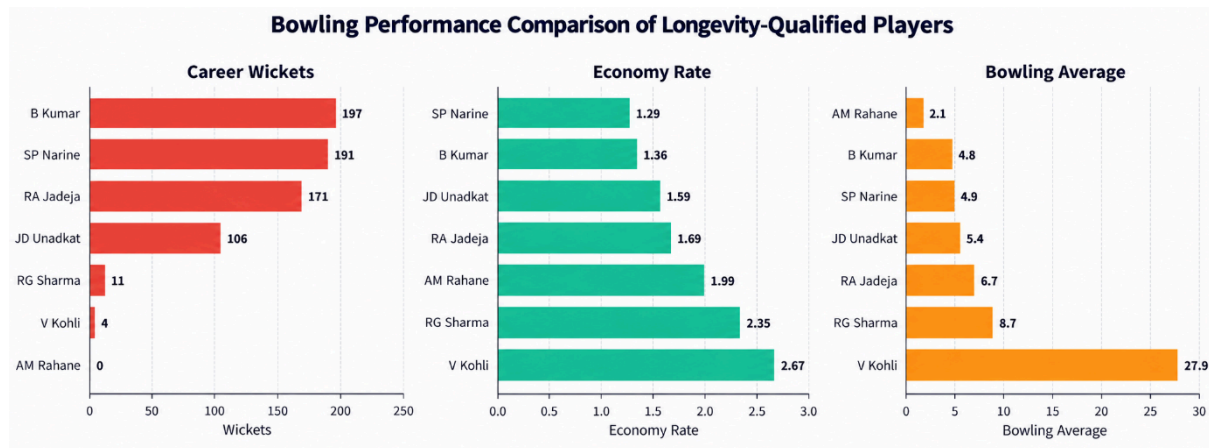


Figure 8.4: Bowling Performance Comparison

B Kumar and SP Narine lead the pool on career wickets, with RA Jadeja and JD Unadkat behind them. On economy rate, SP Narine is again the most efficient, followed by B Kumar; V Kohli is the least efficient of the group, consistent with being a specialist batter who bowls only occasionally.

8.3.4 Cross-Metric Z-Score Patterns

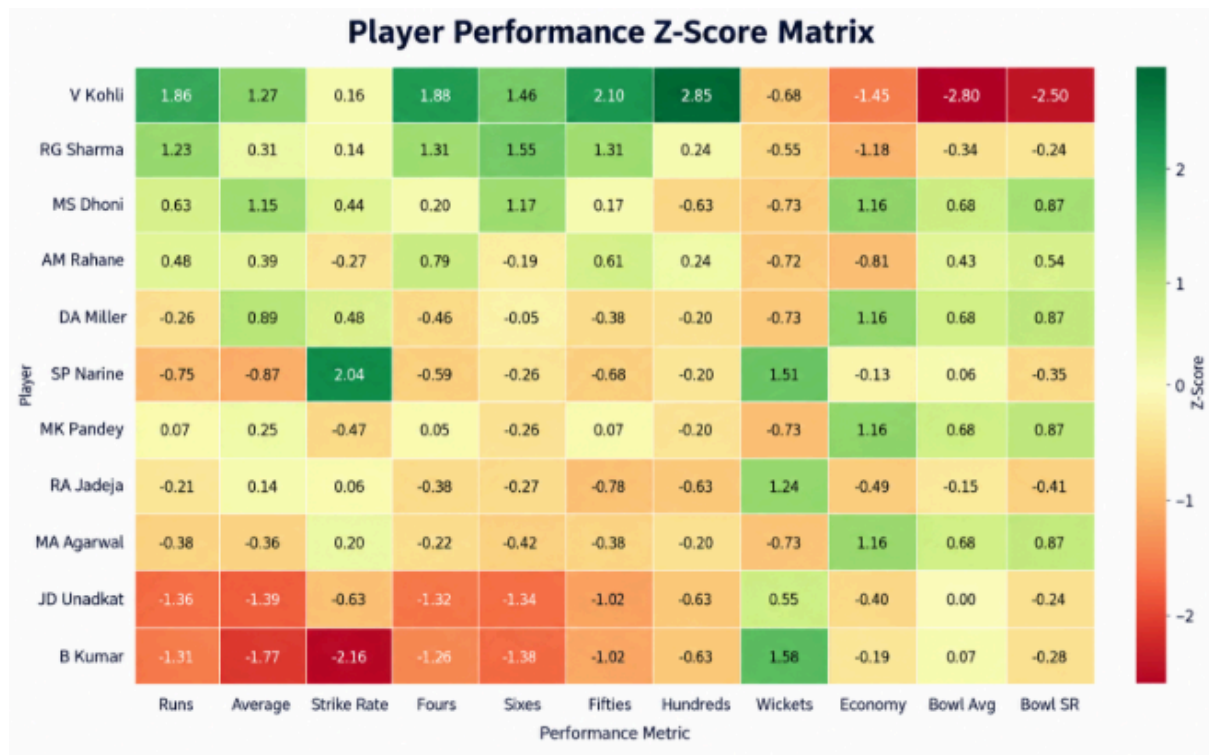


Figure 8.5: Cross-Metric Z-Score Patterns

A combined z-score view across eleven batting and bowling metrics makes two patterns visible that the individual ranked tables leave implicit. First, role specialization shows up as a strong contrast within a single player's own row — Kohli's batting z-scores are almost uniformly strongly positive while his bowling z-scores are strongly negative, and the reverse holds for B Kumar and SP Narine. Second, the specialist bowlers as a group cluster fairly close together on economy and average, while the batting group is more spread out, with Kohli sitting well above the rest of the pool as a clear outlier.

8.3.5 Top-5 Batting Profile

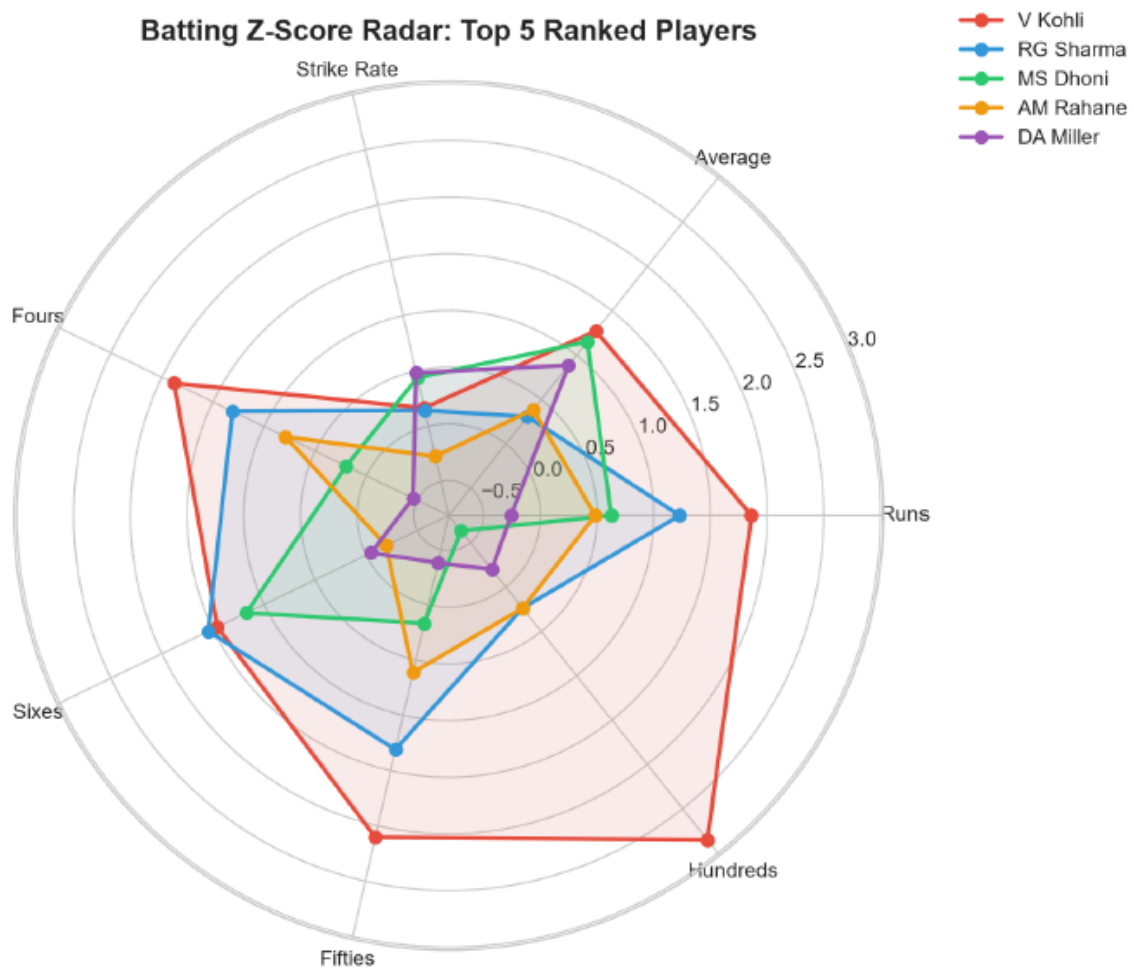


Figure 8.6: Top-5 Batting Profile

Restricting the view to the top five composite-ranked players shows that being top-ranked doesn't mean being a similar kind of player. Kohli dominates on runs, fours, hundreds, and fifties simultaneously — a genuine all-format outlier. Sharma's profile is close to Kohli's on runs and sixes but visibly smaller on hundreds and average, more of a power-hitting profile that trades some consistency for six-hitting output. Dhoni and Miller both show a dip on runs and hundreds paired with a relative peak on average and strike rate — the finisher's profile, where fewer innings are carried to their conclusion but each one is efficient. Rahane's profile is the most balanced of the five but also the lowest-amplitude, reflecting a steady, unspectacular career rather than a standout skill in any one dimension.

8.3.6 Workload vs Output

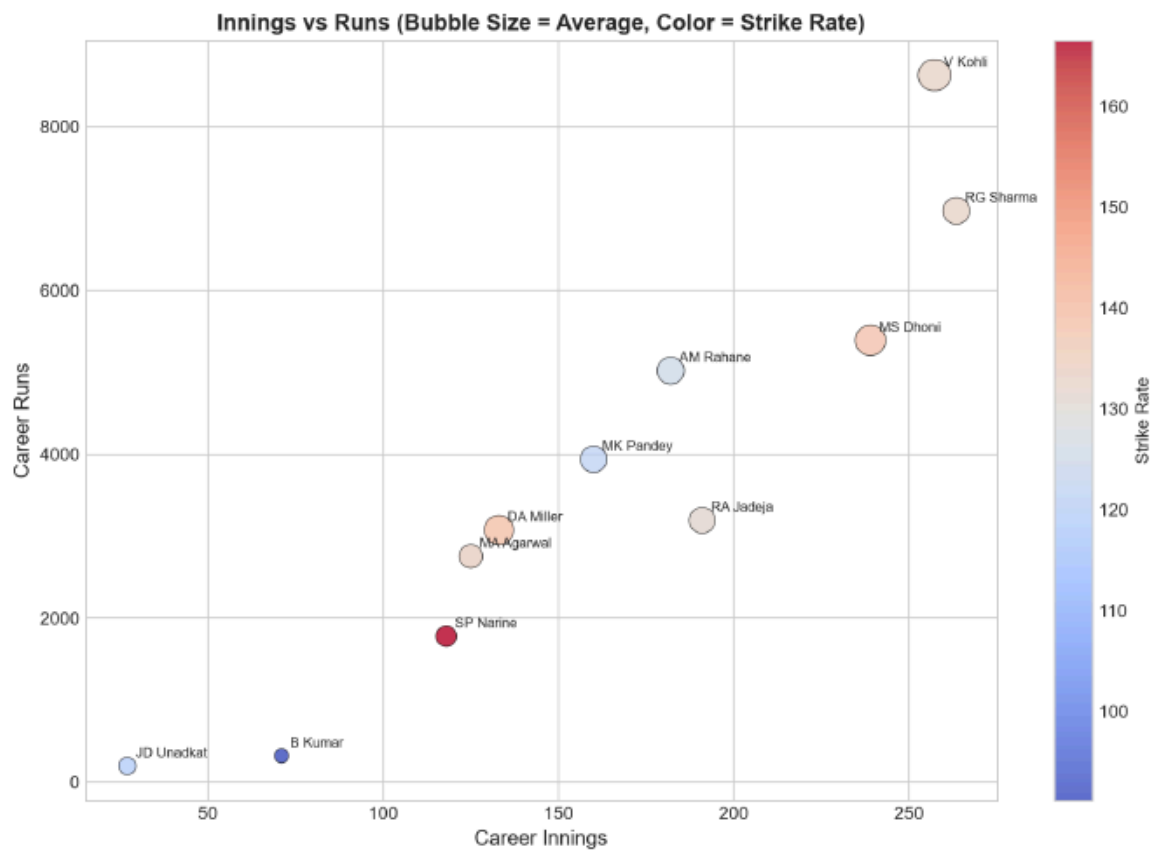


Figure 8.7: Workload vs. Output

Plotting career innings against career runs — with bubble size for average and color for strike rate — separates the pool into three visible clusters: a high-innings, high-runs group (Kohli, Sharma, Dhoni), a mid-tier group (Rahane, Pandey, Jadeja, Miller, Agarwal), and a low-innings specialist-bowler group (Narine, B Kumar, Unadkat). This view is a direct visual argument for the report's core methodological point — raw career totals conflate playing time with quality, and a player's position on a plot like this, not just their total, is what should determine whether they belong in the primary batting pool or should be evaluated on bowling metrics instead.

8.3.7 Quantile-Based Tier Classification

The final classification groups batters into quintiles of batting average and bowlers into quartiles of wickets-per-match.

Table 8.1 — Batting rank and quintile classification

Player	Average	Strike Rate	Innings	100s	50s	Class
V Kohli	39.56	132.89	259	8	63	Class 5 (Top 20%)
MS Dhoni	38.30	137.45	241	0	24	Class 5 (Top 20%)
DA Miller	35.78	138.60	133	1	13	Class 4 (60–80%)
AM Rahane	30.50	125.02	183	2	33	Class 4 (60–80%)
RG Sharma	29.73	132.10	266	2	47	Class 3 (40–60%)
MK Pandey	29.20	121.52	161	1	22	Class 3 (40–60%)
RA Jadeja	27.86	130.30	194	0	5	Class 2 (20–40%)
MA Agarwal	22.97	133.53	125	1	13	Class 2 (20–40%)
SP Narine	17.62	166.51	118	1	7	Class 1 (Bottom 20%)
JD Unadkat	12.31	118.67	27	0	0	Class 1 (Bottom 20%)

Table 8.2 — Bowling rank and quartile classification

Player	Innings	Wickets	WPM	Economy	Bowl Avg	Class
B Kumar	190	198	1.04	1.37	4.85	Class 4 (Top 25%)
SP Narine	187	192	1.03	1.31	4.96	Class 4 (Top 25%)
JD Unadkat	111	110	0.99	1.58	5.43	Class 3 (50–75%)
RA Jadeja	225	170	0.76	1.68	6.68	Class 2 (25–50%)
RG Sharma	32	15	0.47	2.39	9.00	Class 1 (Bottom 25%)

Player	Innings	Wickets	WPM	Economy	Bowl Avg	Class
V Kohli	26	4	0.15	2.65	27.75	Class 1 (Bottom 25%)

The quintile and quartile boundaries are dataset-relative by design (§5.3.3): they self-normalize to whichever pool is supplied, which avoids the empty-tier or single-tier failure modes that fixed numeric cutoffs would produce on a pool this size.

8.3.8 Head-to-Head Player Comparison

Prof. Mehta's feedback after the first review was that ranking players in isolation only gets you so far — a franchise doesn't sit down and ask "where does this player rank out of eleven," it asks "if I'm choosing between these two specific players for this specific slot, who actually wins, and by how much." So on top of the tier classification, we went back and built a head-to-head comparison layer, matching every batter against every other batter and every bowler against every other bowler on the metrics that matter most for each role.



Figure 8.8: Batting Average Head-to-Head Heatmap

A few pairs from the batting matrix make the point better than a general description would. Kohli edges Dhoni by 1.26 on average (39.56 vs 38.30) but is actually behind him by 4.56 on strike rate (132.89 vs 137.45) — so "who's the better batter, Kohli or Dhoni" doesn't really have a single answer; it depends whether the situation calls for accumulation or tempo. Compare Kohli to Miller instead and the gap opens up more: Kohli's ahead by 3.78 on average but behind by 5.71 on strike rate, which tells a similar story but with a bigger swing in both directions — Miller specifically trades average for tempo more aggressively than Dhoni does. The widest strike-rate gap in the whole batting pool is Narine over Unadkat, at 47.84 — which sounds dramatic until you remember Unadkat has 27 career innings against Narine's 118, so at least part of that gap is a small, low-sample batting record for a player who's really in the pool as a bowler.



Figure 8.9: Batting Strike Rate Head-to-Head Heatmap

The bowling matrix tells a tighter story, because the top of that pool is genuinely close. B Kumar and Narine are separated by only 0.06 on economy and 0.11 on bowling average — near enough that either could reasonably be called the pool's best bowler depending on which

single metric you'd rather lean on, and this is exactly the kind of near-tie the tier system alone would flatten into "they're both Class 4" without showing how close it actually is. Jadeja trails Kumar by 1.83 on bowling average despite a very respectable 170 wickets, because his economy and strike rate pull his average up relative to the two specialists ahead of him. And at the other end, Kohli's gap against Kumar is enormous — 22.9 on bowling average alone — which isn't really a knock on Kohli, it's just what the matrix looks like when you put a specialist batter who occasionally bowls next to two players whose entire IPL identity is bowling.

What the heatmap view adds that the ranked tables in §8.3.2 and §8.3.3 don't is the size of the gap, not just its direction. A ranked list tells you Kumar is ahead of Narine on bowling average; the heatmap tells you he's ahead by 0.11, which is close enough that a franchise probably shouldn't treat it as a meaningful difference at all. That distinction — a big gap versus a statistically thin one — is something a rank number alone throws away, and it's the main reason this comparison layer earns its place in the report rather than duplicating what the tier tables already show.

8.4 Results After Implementation

Bringing both studies together highlights the dual importance of methodology and data completeness in this kind of analytics work.

8.4.1 Study 1 Outcomes

- Predictive ceiling: the XGBoost model reached a test R^2 of about 0.54 under honest, time-based evaluation. Predictive accuracy was limited by the absence of causal features in the dataset, not by algorithm choice.
- Dataset gaps: the auction records from 2008–2017 lack retention history, franchise budget signals, injury status, and recent-form indicators — all of which plausibly drive real auction prices and none of which the model could see.
- Bias-variance tradeoff: regularization reduced overfitting but introduced underfitting in its place, reinforcing that the real bottleneck was data coverage, not model architecture.
- Practical impact: we think this model is best used as a sanity-check tool for flagging outlier price expectations, not as a bidding tool — an error margin around ₹14.89M is too wide for tactical use, but it's tight enough to be useful for spotting when a price looks structurally out of line with a player's stats.

8.4.2 Study 2 Outcomes

- Quantile-based classification produced balanced tiers that avoid the distortions fixed numeric thresholds would introduce, evaluating players relative to their peers rather than against arbitrary absolute cutoffs.
- The longevity filter (2012–2025) yielded 11 qualifying players. Extending back to 2008 dropped that to just 4, which says something on its own about how rare sustained top-flight participation actually is.
- Splitting batting and bowling into independent classification tracks correctly recognized elite specialist bowlers like B Kumar and SP Narine, avoiding the undervaluation the composite score produced (§8.3.1, §6.6).
- Practical impact: we think the tiering system is close to directly usable by a franchise for shortlisting and scouting purposes, in a way the price-prediction model currently isn't.

8.4.3 Cross-Study Lessons

Study 2 showed that better methodology applied to existing data — match-level aggregation, rate-based metrics, quantile classification — can produce a strong, defensible output without needing any new data. Study 1 showed the opposite lesson: no amount of preprocessing or regularization can overcome missing causal features. Read together, these two studies suggest that IPL analytics has two genuinely separate levers — methodology and data availability — and that it's worth being clear about which lever a given piece of work is actually pulling on. For a franchise, we'd suggest treating the ranking system as something close to production-usable for shortlisting, and treating the price-prediction model strictly as a diagnostic input to a human decision, not a replacement for one.

Chapter 9: Conclusion and Future Scope

This internship set out to explore how AI-driven decision intelligence could support IPL auction analytics, and produced two concrete, honestly-evaluated studies toward that goal. The price-prediction study shows where data-driven methods currently fall short: predicting an auction price honestly requires information — retention history, franchise intent, recent form, injury status — that simply isn't present in the publicly available auction dataset, and no amount of algorithmic refinement can substitute for it. The player-ranking study shows the more encouraging side of the same coin: a carefully designed, quantile-normalized classification scheme can produce defensible, tier-based leaderboards for batters and bowlers directly from ball-by-ball data, without needing any external variables at all.

Future Scope

- Extend the price-prediction dataset with retention and prior-auction-price history, recent-form indicators, and franchise budget signals — the gaps identified in §5.2 and §6.8.
- Apply the LSTM, ARIMA, and ETS forecasting methods surveyed in the literature review to model player-form trajectories over time, rather than relying on static career averages.
- Explore SMOTE and cost-sensitive learning to better predict rare, high-value (₹15M+) players, which the current model still underestimates.
- Extend Study 2's longevity-qualified pool to a larger, less strict player set, and look at Transformer/BERT-based sequence models for career-trajectory-aware ranking.
- Combine both studies with the optimization-based squad-selection literature — budget, role, and overseas-player-cap constraints — to move from player valuation toward full auction-strategy recommendations, and wrap the pipeline in a dashboard for franchise decision-makers, in line with the internship's original decision-intelligence framing.

References

- [1] X. Schelling and S. Robertson, "A development framework for decision support systems in high-performance sport," *Int. J. Sports Sci. Coach.*, vol. 15, no. 4, pp. 456–468, 2020.
- [2] K. Passi and N. Pandey, "Increased prediction accuracy in the game of cricket using machine learning," *Int. J. Comput. Appl.*, vol. 181, no. 17, pp. 1–6, 2018.
- [3] R. P. Bunker and F. Thabtah, "A machine learning framework for sport result prediction," *Int. J. Comput. Inf. Eng.*, vol. 11, no. 6, pp. 137–144, 2017.
- [4] A. Vaswani et al., "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NIPS)*, Long Beach, CA, USA, 2017, pp. 5998–6008.
- [5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [6] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 2nd ed. Melbourne, Australia: OTexts, 2018.
- [7] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowl. Data Eng.*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009.
- [8] L. Lamas et al., "Optimization models for team selection in high-performance sport," 2015.
- [9] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," in *Proc. AAAI/ACM Conf. AI, Ethics, Soc. (AIES)*, 2017, pp. 1–7.
- [10] N. Mehrabi et al., "A survey on bias and fairness in machine learning," *ACM Comput. Surv.*, vol. 54, no. 6, 2021.
- [11] S. Amershi et al., "Software engineering for machine learning: A case study," in *Proc. ICSE-SEIP*, 2019, pp. 291–300.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

Appendices

Appendix A: Work Log Summary

A phase-by-phase summary of work is provided in Table 3.1 (Chapter 3). Detailed attendance records and daily logs, where maintained separately for institutional purposes, should be attached here as per the guide's requirements.

Appendix B: Data Dictionary (Key Fields)

Field	Description
batter / bowler / non_striker	Player names in each delivery-level role, used to reconstruct (player, season) appearances.
runs_batter, balls_faced	Delivery-level batting outcome fields, aggregated to match level before summing.
wicket_kind, player_out	Dismissal information used to compute career dismissals and batting average.
overs, runs_conceded, wickets	Match-level bowling figures aggregated to career bowling average, economy, and strike rate — see §8.3.3 note on a suspected aggregation issue here.
season, year	Used to build the longevity-qualification filter (appearance in every season 2012–2025).
Status, LMatches, LRuns, LSixes, LWkts, LFours	Auction-record features used in the XGBoost price-prediction model (Study 1).